



PDF Download
3718958.3754342.pdf
10 January 2026
Total Citations: 0
Total Downloads: 2166

Latest updates: <https://dl.acm.org/doi/10.1145/3718958.3754342>

RESEARCH-ARTICLE

Unlocking Superior Performance in Reconfigurable Data Center Networks with Credit-Based Transport

FEDERICO DE MARCHI, Max Planck Institute for Informatics, Saarbrücken, Saarland, Germany

JIALONG LI, Shenzhen University of Advanced Technology, Shenzhen, Guangdong, China

YING ZHANG, Meta, Menlo Park, CA, United States

WEI BAI, NVIDIA, Santa Clara, CA, United States

YITING XIA, Max Planck Institute for Informatics, Saarbrücken, Saarland, Germany

Open Access Support provided by:

Max Planck Institute for Informatics

Shenzhen University of Advanced Technology

Meta

NVIDIA

Published: 08 September 2025

[Citation in BibTeX format](#)

SIGCOMM '25: ACM SIGCOMM 2025 Conference

September 8 - 11, 2025
Coimbra, Portugal

Conference Sponsors:
SIGCOMM

Unlocking Superior Performance in Reconfigurable Data Center Networks with Credit-Based Transport

Federico De Marchi
Max Planck Institute for Informatics

Jialong Li*
Shenzhen University of Advanced
Technology

Ying Zhang
Meta

Wei Bai
NVIDIA

Yiting Xia
Max Planck Institute for Informatics

Abstract

The large-scale, end-to-end implementation of microsecond-switched reconfigurable data center networks (RDCNs), coupled with innovative routing and topology designs that provide continuous routes abstracting away frequent topology changes, demonstrates promise as a viable alternative to Clos networks in the post-Moore's Law era. However, the gap remains in transport performance, with current transport solutions falling short of unlocking their full performance. In this paper, we introduce Flare, a novel credit-based transport protocol that ensures reliable traffic delivery, low latency, and leverages the rapidly reconfiguring circuits of the RDCN to opportunistically route traffic over short paths, maximizing throughput. In simulations, Flare enables RDCNs to outperform Clos networks, achieving up to 1.15× higher throughput even under adversarial traffic. Additionally, it delivers up to 2× and 1.5× higher throughput than NDP and ExpressPass, and up to 10×, 15×, and 3.5× shorter flow completion time (FCT) than ExpressPass, TDTCP, and Bolt. Our testbed implementation further demonstrates the feasibility of Flare's mechanisms with programmable switches and DPDK.

CCS Concepts

• Networks → Transport protocols; Data center networks.

Keywords

Transport protocols, congestion control, data center networks, reconfigurable data center networks

ACM Reference Format:

Federico De Marchi, Jialong Li, Ying Zhang, Wei Bai, and Yiting Xia. 2025. Unlocking Superior Performance in Reconfigurable Data Center Networks with Credit-Based Transport. In *ACM SIGCOMM 2025 Conference (SIGCOMM '25)*, September 8–11, 2025, Coimbra, Portugal. ACM, New York, NY, USA, 19 pages. <https://doi.org/10.1145/3718958.3754342>

1 Introduction

The slowdown of Moore's Law for merchant silicon has driven the development of reconfigurable data center networks (RDCNs) [3, 7, 8, 10–12, 16, 18, 27, 30–32, 37, 43]. As illustrated in Fig. 1, RDCNs

leverage high-capacity and power-efficient optical circuit switches (OCSeS) to interconnect electrical endpoints such as pods, Top-of-Rack switches (ToRs), and host NICs. Unlike traditional data center networks (DCNs), RDCNs dynamically construct optical circuits in the OCSeS to create flexible network topologies.

The inherent difficulty of achieving both low *reconfiguration delay* and high *port count* in OCSeS motivated the networking community to navigate through different tradeoffs in RDCN designs. As shown in the spectrum of reconfiguration frequency (Fig. 2), industrial solutions, represented by Google's Jupiter [39] and Light-wave [29] fabrics, transitioned gradually from static topologies to use OCSeS as cabling patch panels, reconfiguring at hour-to-day intervals to adapt to long-term traffic patterns. This approach allows the use of high-radix OCSeS and aligns with the conventional practice of operating the DCN still as a static topology (between occasional reconfigurations), but requires a large number of circuits to maintain network connectivity at all times—unlike time-shared circuits in fast-switching designs.

Academia, in contrast, pursues more radical innovations for performance gains. Researchers first proposed *traffic-aware* RDCNs that dynamically establish circuits to match instantaneous DCN traffic, at hundreds of milliseconds to seconds frequency constrained by both the speed of the OCSeS used and the time required for global traffic collection and topology computation. The resulting delay either significantly impacts flow completion [10, 11, 37] or necessitates a redundant static electrical DCN—the so-called electrical-optical hybrid architecture—to ensure continuous connectivity during reconfigurations [16, 43], at considerable cost, complexity, and power overhead in deploying dual fabrics.

This has pushed for ever-reducing reconfiguration time, from microseconds to nanoseconds, with advanced OCS technologies and thus the emergence of *traffic-oblivious* RDCNs [3, 7, 30, 32]. At these timescales, these architectures avoid the time-consuming control loops by cycling through a pre-defined series of network topologies, ensuring diverse connectivity regardless of traffic patterns. Despite their performance potential, the challenges of scaling OCS manufacturing and the disruption caused by frequent reconfigurations to the network stack continue to hinder their adoption.

The recent large-scale implementation of Opera with commodity devices [31] marks a milestone in the real-world deployment of traffic-oblivious RDCNs. Microsecond-switching OCSeS have been manufactured at hundreds of ports [31], and Opera's topology-routing codesign staggers circuit reconfigurations to provide continuously connected paths between all nodes at all times [30]. For the first time, it preserves the routing abstraction of continuous

*Work done at Max Planck Institute for Informatics.



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

SIGCOMM '25, Coimbra, Portugal

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1524-2/25/09

<https://doi.org/10.1145/3718958.3754342>

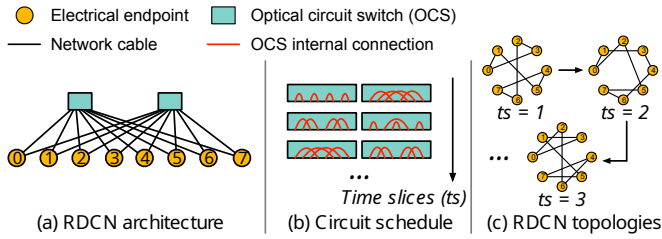


Figure 1: Illustration of a traffic-oblivious RDCN.

connectivity in static networks, enabling DCN applications to run seamlessly on reconfigurable fabrics without interruption.

Yet, one cornerstone remains for traffic-oblivious RDCNs to reach their full potential: *transport performance*. At equivalent equipment cost, expander graphs in Opera provide higher bisection bandwidth than Clos networks, thanks to their smaller network diameter and denser connectivity [23, 42, 45]. However, Opera’s native transport protocol, NDP [20], originally designed for non-blocking Clos networks, achieves only half the throughput of Clos due to poor adaptivity to RDCNs. Similarly, TDTCP [13], a transport protocol tailored for electrical-optical hybrid RDCNs, is mismatched for traffic-oblivious RDCNs with purely optical core network fabrics and performs no better than vanilla TCP [15].

In this paper, we *bridge the performance gap in traffic-oblivious RDCNs* with **Flare**, a *dedicated transport protocol for Opera*—the most readily deployable traffic-oblivious RDCN architecture. Our goal is to elevate Opera’s transport performance beyond that of traditional Clos-based DCNs, even under the worst-case traffic, establishing traffic-oblivious RDCNs as a scalable, power-efficient, and high-performance foundation for cloud networks in the post-Moore’s Law era.

Flare’s design is tailored to unique characteristics of Opera: (1) frequently reconfigured topologies each lasting only microseconds, or a few RTTs; (2) the lack of equal-cost parallel paths in expander graphs, unlike in fully provisioned Clos networks, causing congestion to be commonplace in Opera’s core fabric, even with packet spraying; and (3) the rotating topologies providing relatively uniform connectivity across node pairs per reconfiguration cycle.

Through a comprehensive survey of transport techniques (§3.1), we identify *credit-based transport* as a good fit for (1) and adopt it in Flare. The constant shifts in bottleneck bandwidth and location, driven by topology and routing changes, render reactive protocols, reliant on “try and backoff” bandwidth probing, ineffective. In contrast, proactive bandwidth allocation via credits ensures fast adaptation to reconfiguration, which is why Opera initially adopted NDP. However, most credit-based and other proactive schemes assume rearrangeably non-blocking networks, failing to handle *multi-bottlenecks* in (2) beyond the sender NIC and receiver ToR downlink. Flare, by contrast, shapes credits across all links, essential to prevent data paths from becoming overwhelmed.

The key challenge in Flare is hereby credit waste from link-wise credit shaping: when a credit packet is dropped at a later bottleneck, the bandwidth it reserved at earlier links is wasted and unavailable to other flows. This is particularly costly in RDCNs, where circuits are only briefly available. Unfortunately, no existing solution directly addresses this. ExpressPass [14], the only credit-based

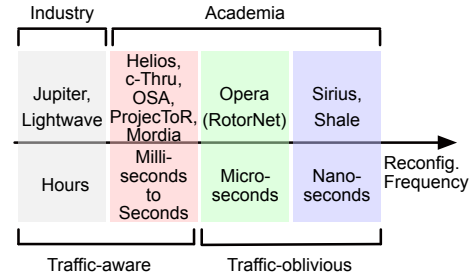


Figure 2: RDCNs of different reconfiguration frequencies.

protocol designed for multi-bottlenecks DCNs, uses a continuous feedback loop to adjust flow credit rates based on credit loss. Like reactive transport, this approach fails to converge in RDCNs.

In Flare, we leverage a key insight from (3) to tackle this challenge. *Fairness* in bandwidth sharing is a key concern in traditional DCNs, but in Opera, enforcing fairness across fleeting topology instances is both impractical and unnecessary. Instead, we relax this constraint to maximize *utilization*, efficiently use scarce circuit resources. Since credits traversing more hops are more likely to be dropped and contribute to credit waste, favoring flows with shorter paths¹ improves utilization. At each bottleneck, credits are admitted inversely proportional to their remaining hop count, prioritizing flows with fewer remaining hops. While this *opportunistic* strategy sacrifices instant fairness, (3) ensures all source-destination pair have relatively equal chances to benefit from short paths over a cycle, offering *long-term fairness* at cycle level.

Besides this main design insight, Flare addresses additional issues such as mismatched credit and data paths across topologies, slow convergence to optimal credit rates in dynamic topologies, and underutilization of credit paths due to topology and traffic dynamics. We implement Flare on our testbed using DPDK and Tofino2 programmable switches, and evaluate it in large-scale simulations with production DCN traces. In our 108-ToRs simulated Opera network, Flare achieves up to 2× and 1.5× higher throughput than NDP and ExpressPass, and up to 10×, 15×, and 3.5× shorter FCTs compared to ExpressPass, TDTCP, and Bolt. Notably, Flare enables Opera to outperform a similar-cost Clos network, delivering up to 1.15× higher throughput. We demonstrate Flare’s functionality on our testbed with cross-validation of the simulation results.

Customized to Opera for practicality, Flare is not intended for nanosecond-reconfigured traffic-oblivious RDCNs like Sirius [7] and Shale [3], which rely on fundamentally different hardware architectures and routing mechanisms (§2.1) and remain far from end-to-end realization. Flare aims at immediate deployment of traffic-oblivious RDCNs and is orthogonal to theoretical studies on transport for nanosecond-scale architectures [3, 4, 44].

[This work does not raise any ethical issues.]

2 Background

In this section, we briefly introduce traffic-oblivious RDCNs (§2.1), with particular focus on Opera (§2.2), to provide context for Flare specifically designed for this setting.

¹The name *Flare* is inspired by the phenomenon that lightning follows the path of least resistance.

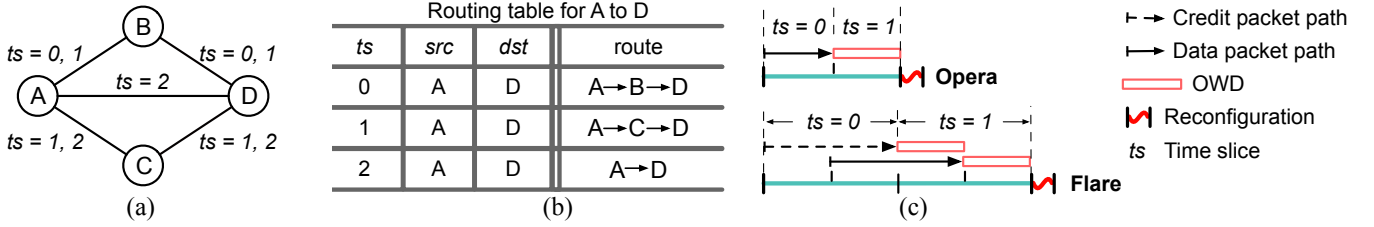


Figure 3: Opera's (a) staggered circuit reconfigurations, (b) time-based routing, (c) bounded OWD for lossless transmission.

2.1 Traffic-Oblivious RDCNs

As introduced in §1, traffic-oblivious RDCNs are a recent trend in RDCN design. Since they are the focus of this paper, we refer to them simply as RDCNs unless otherwise specified.

Fig. 1 illustrates a RDCN, where a reconfigurable fabric, typically composed of OCSes, connects electrical communication endpoints such as pods, ToRs, or host NICs. Endpoints have limited optical uplinks to the OCSes, e.g., single or dual ports per NIC, and several uplinks per switch. A *network controller* reconfigures the OCSes at fixed intervals, called *time slices*, during which optical circuits form unique network topologies. Over time, circuits are reconfigured according to a pre-determined *circuit schedule*, usually variants of *round-robin* connections across nodes, repeating every *configuration cycle*, regardless of the traffic pattern. A cycle typically consists of tens to hundreds of time slices.

Traffic is routed via *direct circuits* between source and destination or through *multi-hop paths* that make use of available circuits [3, 7, 32]. A common strategy is valiant load balancing (VLB) [7, 32], and its variants [3], which route each packet via a random intermediate node to exploit circuit diversity and maximize throughput. We refer to these approaches as *discontinuous path routing*, where packets wait—at source or intermediate nodes—for the next scheduled circuit, and routing latency is dictated by this waiting time.

To reduce routing latency, recent architectures, such as Sirius [7] and Shale [3], have pushed time slice durations to tens of nanoseconds. However, such fine-grained reconfigurations face significant challenges in scaling nanosecond-speed OCSes and adapting the network stack. To date, the only large-scale implementation of traffic-oblivious RDCNs that has been demonstrated in practice is Opera [31], which operates with time slices in the tens of microseconds. As discussed in §2.2, Opera ingeniously retains *continuous path routing*, as in traditional static networks, to mask the reconfiguration delay while remaining manufacturable at scale.

2.2 Opera

Opera [30] builds on the RotorNet architecture [32], which demonstrated microsecond-reconfigured OCSes with hundreds of ports [31], and extended it into a complete system. In this paper, we refer to RotorNet as the optical architecture and Opera as the full network stack on top of it. Opera stands out from other traffic-oblivious RDCNs for its implementability, achieved by its unique preservation of continuous connectivity over the reconfigurable network fabric through tight coordination of *topology*, *routing*, and *transport*.

To maximize network connectivity, Opera's topology goes beyond simple round-robin circuit schedules, employing a sequence of time-varying expander graphs, which we refer to as *varying expanders*. These varying expanders are meticulously designed to

ensure that: (1) each node maintains a round-robin connection to all other nodes, but in a permuted order, preserving uniform pairwise connectivity across a reconfiguration cycle; and (2) circuit reconfigurations are staggered, with each circuit lasting N time slices and only reconfigured at the end, so that at any given time, only a $\frac{1}{N}$ fraction of circuits change. With these, the Opera topology provides **continuous connectivity for all source-destination pairs at all times**, utilizing the stable circuits that remain unreconfigured.

Continuous connectivity is achieved through seamless path transitions enabled by *time-based routing*. Given a fixed circuit schedule, Opera precomputes *multi-hop*² routes for each source-destination pair on every expander instance and preloads them onto the source endpoints, i.e., switches or host NICs connected to the optical fabric, enabling source routing. All endpoints sync with the network controller to accurately identify time slices. As shown in Fig. 3b, Opera's routing table includes a time slice match field. Upon packet arrival, the source determines the current time slice and routes the packet along the corresponding precomputed path, seamlessly switching paths in the final time slice before circuit reconfiguration, without disrupting packet flow.

Lossless transmission in the optical fabric requires **Opera's time slice duration be strictly larger than the one-way delay (OWD)**. As shown in Fig. 3c, packets can reliably transmit up to the network's OWD before reconfiguration, after which in-flight packets risk being lost. Thus, the final time slice during which packets are redirected must fully cover the OWD. OWD is predictable only if queuing delays are bounded. Opera achieves this at the transport layer with NDP [20], leveraging packet trimming to strictly limit queue sizes, and sets the time slice duration to this bounded OWD.

Fig. 3 shows an example with $N = 2$, where each path persists for two time slices. As in Fig. 3a, the path A → B → D is physically active in time slices $ts = 0$ and $ts = 1$, but it can only be used reliably in $ts = 0$, as packets remaining on the path in $ts = 1$ (one time slice or OWD before reconfiguration in $ts = 2$) risk loss. Thanks to Opera's staggered circuit reconfiguration, in $ts = 1$, another path A → C → D becomes available. The routing table in Fig. 3b enforces the path transition. Similarly, this new path is up in $ts = 1$ and $ts = 2$ but only usable in $ts = 1$, so in $ts = 2$ packets must be transitioned to yet another path A → D.

3 Problem Formulation

Opera's network design differs fundamentally from traditional DCNs and is unique even among RDCNs (§2). As introduced in §1, the major factor limiting Opera's performance in the current

²Opera includes an *offload* mode to offload heavy flows to RotorNet's original VLB routing and RotorLB transport [32]. Flare is designed for the *default* multi-hop routing and coexists with the offload mode as needed.

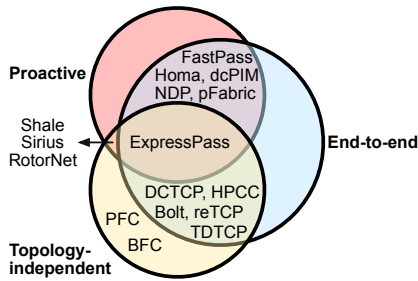


Figure 4: Venn Diagram of existing transport solutions.

state of the art is *transport performance*: no proposed protocol fully unlocks its potential. This is because, despite the circuit-switched network core, congestion in RDCNs still arises at the edge electrical endpoints (Fig. 1), and is hard to manage due to the dynamic nature of the network. This is especially true in Opera’s varying expander, where the electrical endpoints also serve as transit nodes for multi-hop paths.

We formulate this new problem to enable a systematic design of Flare, first surveying the adaptability of existing transport protocols to Opera’s varying expander to inspire our work (§3.1) and then identifying the specific challenges facing Flare (§3.2).

3.1 Transport on Varying Expanders

As a literature review, we categorize existing transport solutions along three dimensions, illustrated in Fig. 4. We argue that unique characteristics of varying expanders (§2.2) request a protocol simultaneously meeting these criteria, and we clarify that existing transport proposals for RDCNs are designed for different RDCN settings and mismatched for Opera.

Proactive. The most widely used *reactive* transport [1, 5, 24, 26, 33, 46], due to its “try and backoff” nature, struggles to adapt to constantly changing bottleneck bandwidth and locations in varying expanders with microsecond-scale time slices, where each topology lasts merely a few RTTs. During reconfiguration, uncleared packets in egress queues exit to wrong or non-existent circuits, leading to routing loops or packet drops. Conversely, *proactive* [2, 9, 36], predominantly *credit-based* [14, 20, 34] methods regulate transmission rates to maintain high throughput and low queue occupancy, which eliminates the need for bandwidth probing and ensures bounded queues, satisfying the OWD requirement in Opera (§2.2). Essentially, this is why Opera adopted NDP.

TDTCP [13] and reTCP [35] still employ reactive transport, namely TCP variants, because they are designed for electrical-optical *hybrid* RDCNs. They address bandwidth fluctuations of long flows transitioning between the low-bandwidth electrical and high-bandwidth optical fabrics, while short flows stay within the electrical network, unlike Opera, which carries all the traffic over its purely optical fabric.

They adapt poorly to Opera for this reason. Specifically, reTCP pre-buffers packets and enlarges cwnd for rapid ramp-up when high bandwidth becomes available, but fails to handle Opera’s uniform bandwidth across time slices with varying topologies. TDTCP maintains separate TCP state machines for electrical and optical networks, but this approach does not scale from two (electrical vs. optical) time slices in hybrid architectures to tens to hundreds of time slices in Opera (§2.1). Due to the prevalence of short flows, few

flows persist beyond a configuration cycle. As a result, TCP states associated with a given time slice are either single-use or obsolete by the next cycle due to drastic shifts in network conditions.

Topology-independent. Almost all proactive transport protocols require the entire network [2, 9, 20, 34] or individual tiers [36] to be *rearrangeably non-blocking*. This property underpins precise per-packet bandwidth allocation [2, 9] and enables credit-based solutions [20, 34] to assume a congestion-free core with packet spraying—sufficient to control traffic at the sender NIC and receiver ToR downlink only. While this assumption holds in fully provisioned, non-blocking Clos networks, it does not in varying expanders, where the lack of equal-cost parallel paths renders core congestion inevitable, even with packet spraying. Consequently, protocols that fail to handle *multi-bottlenecks*, such as Opera’s original NDP, perform poorly. Thus, Flare requires a more general protocol that extends beyond the traditional Clos topology.

End-to-end. The strict OWD requirement (§2.2) makes *end-to-end* protocols a natural fit for Opera. In contrast, *hop-by-hop* solutions, such as PFC [22] and BFC [19], that use pause frames to control traffic on individual links struggle to bound queuing and queue-pausing delays, which together contribute to the overall OWD. Without a global view of multi-bottlenecks in varying expanders and given the complex interplay of numerous flows under Opera’s multi-hop routing, it becomes extremely challenging, if not impossible, to precisely pause queues network-wide such that (1) the paths remain unchanged when paused queues are released, and (2) all queues are fully cleared by reconfiguration time. Any violation of these conditions results in packets being forwarded to circuits that are either down or misdirected, leading to packet loss in the optical fabric.

Sirius [7], Shale [3], and RotorNet [32], on the other hand, use hop-by-hop credit-based transport tightly coupled with *hop-by-hop VLB routing* (§2.1). In these systems, packets are buffered for one or more time slices while awaiting tokens from the next hop—intrinsically incompatible with Opera’s *continuous* end-to-end packet delivery within a single time slice (the OWD). As explained in §2.1, the core limitation of VLB is the circuit-waiting delay, now exacerbated by the multi-round token exchange. Consequently, Sirius and Shale pursue nanosecond-scale time slices—a challenging implementation goal; and the microsecond-scale RotorNet offloads latency-sensitive flows to a parallel electrical network (i.e., the hybrid architecture), ultimately motivating Opera’s shift to continuous path routing. Even decoupled from VLB, a hop-by-hop transport on Opera would suffer the same issues as PFC/BFC, making truly lossless transmission untenable.

3.2 Challenges of Flare

As shown in Fig. 4, ExpressPass [14] is the only protocol at the intersection of the above criteria (§3.1). Rather than designing Flare from scratch, we draw inspiration from ExpressPass, the closest solution for static DCNs.

ExpressPass addresses multi-bottlenecks by pacing credits issued by receivers across *all* links and allowing senders to transmit an up-to MTU-sized data packet only upon receiving a credit. Credit packets traverse rate-limited queues at each NIC or switch to ensure the amount of corresponding data packets remain within link

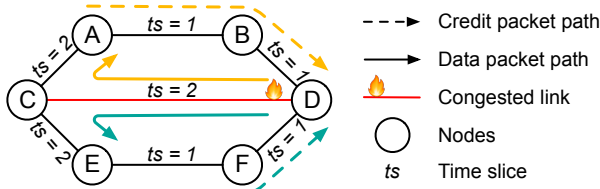


Figure 5: Asymmetric credit-data paths in varying expanders.

capacity. However, as detailed below, ExpressPass' success depends on two critical requirements that varying expanders do not satisfy. Flare's design directly addresses these challenges.

Challenge 1: credit and data packets sent across different time slices may traverse asymmetric paths, invalidating the credit allocation.

Each credit reserves a specific end-to-end path under multi-bottlenecks. To maintain consistent bandwidth allocation, it is crucial for data packets to follow the reverse path of their corresponding credits, requiring *symmetric credit-data paths* between each source and destination.

Varying expanders violate this requirement. As shown in Fig. 5, during time slice 1 ($ts = 1$), credit packets from receiver A to sender D are allocated for path $A \rightarrow B \rightarrow D$ and credit packets from receiver E to sender D are allocated for path $E \rightarrow F \rightarrow D$. By the time these credits reach D, the topology has shifted to $ts = 2$, where the data path from sender D to receiver A becomes $D \rightarrow C \rightarrow A$, and from sender D to receiver E becomes $D \rightarrow C \rightarrow E$. Unaware of the change, data packets are sent at full bandwidth for the original paths, causing congestion on the now-overlapping link $D-C$.

Challenge 2: credit waste along multiple bottlenecks cannot be minimized through a continuous feedback loop, causing low link utilization.

With credit shaping at all links, each successfully transmitted credit reserves a unit of bandwidth per link it traverses. Multi-bottlenecks, like in Opera's varying expanders, inevitably cause *credit waste*: when a credit packet traverses several links and finally dropped, it consumes multiple bandwidth units that could have been allocated to other flows.

For example, in Fig. 6a, three credit flows compete for bandwidth across two links. Assuming each aims for 100% data rate, flows $A \rightarrow B$ and $A \rightarrow C$ share the bandwidth of link $A-B$ equally at 50% each. On link $B-C$, $A \rightarrow C$, now reduced to 50% bandwidth, competes with flow $B \rightarrow C$, still at 100%, resulting in allocations of 33% and 67%, respectively. Because of the two bandwidth bottlenecks, flow $A \rightarrow C$ has wasted $50\% - 33\% = 17\%$ bandwidth, underutilizing link $A-B$.

In static networks, this issue is addressed via a continuous *feedback loop* that adjusts each flow's credit rate based on lost credits. As shown in Fig. 6b, all flows reduce their credit rates in response to credit loss and gradually increase them to probe available bandwidth. Since lower-rate flows increase more aggressively, after several RTTs of adjustments, the rates converge to a fair 50% bandwidth share, achieving full link utilization. In contrast, varying expanders prevent such convergence due to constant topology changes, resulting in credit waste in the initial state of Fig. 6a or, at best, partial progress in some intermediate state towards Fig. 6b.

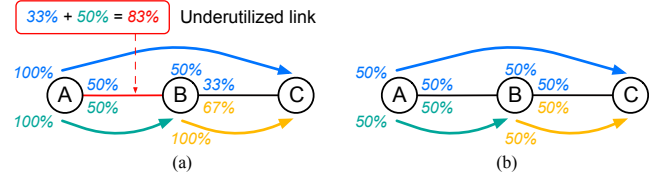


Figure 6: (a) Credit waste in varying expanders vs. (b) no credit waste in static networks.

4 Design

At a high level, Flare is a credit-based transport protocol that controls congestion at multiple bottlenecks performing credit pacing at all links, like ExpressPass [14], but is specifically designed to tackle the challenges of varying expanders (§3.2).

Flare addresses **Challenge 1** in §4.1 by enforcing strict symmetry between credit and data paths, even as the topology changes. This is fundamental to the overall design. **Challenge 2** concerns low link utilization due to high credit waste in the varying expander, and is tackled through §4.2–§4.4. The key insight of Flare is that, on the varying expander, credit admission can be made *probabilistic* (§4.2), which steers bandwidth allocation towards shorter paths to increase utilization and reduce credit waste. We then further extend the original ExpressPass credit-rate control (§4.3), and introduce a tentative credit system (§4.4) to fill potentially unused bandwidth. Further optimizations are discussed in §4.5.

As illustrated in Fig. 7, Flare uses two types of credit packets—*regular* and *tentative* credits—which traverse credit queues at each link. Each credit queue has three thresholds based on the number of credit packets they hold. Tentative credits are only admitted into the queue below the very low *tentative credit threshold*. The *congestion threshold* is for shaping regular credits in presence of contention: below it, all regular credits are admitted, but above it, admission becomes probabilistic (§4.2). Finally, any credits exceeding the *maximum credit queue size* are always dropped.

4.1 Credit-Data Path Symmetry

To address Challenge 1 of varying expanders (§3.2), we enforce symmetric credit and data paths with minimal changes to Opera's circuit reconfiguration and routing.

Recall from §2.2 that Opera sets the time slice duration to OWD, allowing reliable packet transmission over a path until one time slice (or OWD) before it is taken down. In comparison, as illustrated in Fig. 3c, Flare requires the exchange of both data and credit packets. Data packets can only be transmitted after the sender receives the first credit packet from the receiver, which takes OWD. Therefore, in Flare, we double the time slice duration to 2OWD, or simply RTT, to accommodate the full round trip for both the credit and data packets. In this way, we extend the path availability to let data packets follow the reverse path of the credit packets.

However, Opera's routing table is unaware of the change of time slice duration and continues using paths until one time slice (this time RTT not OWD) before reconfiguration. As shown in Fig. 3c, since data packets lag behind credit packets by OWD, their transmission spans OWD into the last time slice before reconfiguration ($ts = 1$ in this example). The routing table (Fig. 3b) would then direct them to the follow-up path in $ts = 1$ (Fig. 3a)—the reverse

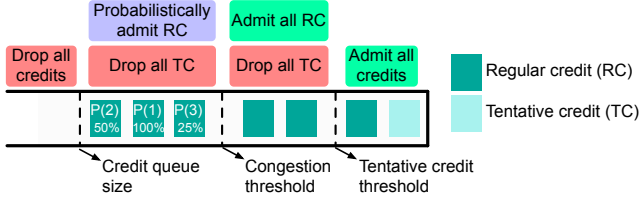


Figure 7: Flare's thresholds for regular and tentative credits.

path of $A \rightarrow C \rightarrow D$, instead of $A \rightarrow B \rightarrow D$ in $ts = 0$ —breaking the symmetry.

We modify Opera's time-based routing (§2.2) to achieve path symmetry. Opera relies on each packet's time slice entering the optical fabric (at the source endpoint switch or NIC) for source routing. We embed this time slice into credit packets and perform routing lookup based on this embedded field. Upon receiving a credit, the corresponding data packet inherits its time slice, pretending to arrive OWD earlier. This change offsets time-based routing for data packets by OWD, and still guarantees lossless transmission, as data packets stop traversing the route OWD before reconfiguration.

This simple modification appears to double the time slice duration from OWD in Opera to RTT, but in practice, as shown in our simulation §6.3, Flare achieves negligible queuing on data paths with precise credit-based scheduling, enabling shorter time slice durations than before without sacrificing performance. The short time slice durations ensure flexibility for circuit reconfiguration, and Opera's circuit utilization is preserved by maintaining the same staggered circuit reconfiguration pattern (§2.2), where only a $\frac{1}{N}$ fraction of circuits undergo reconfiguration at any given time.

4.2 Probabilistic Credit Admission

As Challenge 2 (§3.2) highlights, minimizing credit waste exclusively via credit-rate limiting on the varying expander would require precise prediction of ideal credit allocation per link—something infeasible due to complex flow interactions.

In Flare, we take advantage of the rotating topologies of the varying expander to design a new credit admission system with low waste, even when the credit rate cannot be precisely adjusted. We introduce a *stateless probabilistic credit admission* heuristic based on a simple insight: credit packets over longer paths are more likely to be dropped at multiple bottlenecks, wasting credits along the way. At each bottleneck, the heuristic favors admission of credit packets with *fewer hops remaining*, as they are statistically less likely to encounter future contention, thereby reducing overall credit waste. This heuristic steers credit allocation towards short paths and trades fairness for enhanced circuit utilization.

This tradeoff aligns with the *long-term fairness* Opera offers: as in §2.2, the circuit schedule features uniform connectivity between nodes across a reconfiguration cycle, giving source-destination pairs relatively equal opportunities for short paths on the cycle scale. Our evaluation in §6.2 shows near-optimal fairness at the cycle level. We argue this is sufficient for long flows, the primary concern for fairness, given Opera's microsecond-scale circuit durations, which result in short configuration cycles lasting only milliseconds.

Specifically, as illustrated in Fig. 7, when a credit queue is below its *congestion threshold* and bandwidth competition is low, all regular credit packets are accepted to avoid unnecessary drops. Without

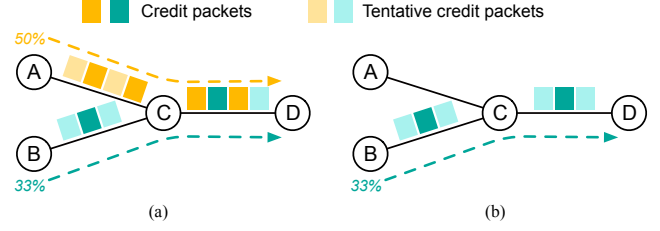


Figure 8: Tentative credits to increase link utilization.

this threshold, a single flow with no competition but multiple hops remaining could experience unnecessary credit drops, resulting in throughput loss.

$$P(h) = \left(\frac{1}{2}\right)^{h-1} \quad (1)$$

Probabilistic admission is triggered once the congestion threshold is crossed: at this point the queue accepts credit packets based on the above probability rate P , where h represents the credit packet's remaining number of hops embedded in the packet header. We have tested various admission functions (§6.2), and found that the exponential function in Eqn. 1 yields minimum credit waste without excessive credit dropping from longer paths.

4.3 Credit Rate Control

Probabilistic credit admission (§4.2) is our primary performance driver, differentiating flows based on their path lengths when shaping credits. In another dimension, it is also important for the receiver to send credits at appropriate target rates, feeding flows with sufficient credits when their admission probabilities are high to boost utilization, and reducing the rate in response to credit drops to minimize waste.

However, this can only be best-effort in the varying expander due to the convergence difficulty outlined in Challenge 2 (§3.2). We adapt the credit rate control mechanism from ExpressPass, taking advantage of its feedback loop that adjusts based on credit loss over time, and make the following adjustments to better suit Opera.

First of all, in contrast to ExpressPass, flows start at an aggressive credit rate of 100%. This allows flows to seize bandwidth quickly, without necessarily incurring heavy waste thanks to probabilistic admission. Then, inspired by TDTCP [13], which maintains separate states for different topology configurations, we reset the credit loss count entering a new time slice. Yet, rather than preserving the target rate per time slice for reuse in the next cycle—since, as pointed out in §3.1, few flows persist for an entire cycle—we carry over the rate to the new time slice and make rapid adjustments to align with the new topology configuration.

Upon a configuration change, flows that are subject to a path change may need to adjust their rate according to their new path length. When a flow transitions from a longer path to a shorter path, the rate should be ramped up to seize the additional bandwidth that comes with higher probability admission. We increase the rate by $P(\text{new_path_length}) - P(\text{old_path_length})$ to compensate for the difference in credit admission probabilities in Eqn. 1. This adjustment enables the flow to immediately take advantage of the potential additional bandwidth without needing possibly several RTTs to ramp up. Conversely, when transitioning from a shorter path to a longer one, it is not necessary to immediately

decrease the rate. This is because our probabilistic credit admission (§4.2) is designed to naturally drop excessive credits if the flow begins contending with others on shorter paths. As a result, the flow is unlikely to cause waste, even in the presence of multiple bottlenecks.

4.4 Tentative Credits

Despite these efforts, circuit underutilization may still occur due to undershooting in best-effort credit rate control (§4.3) and network dynamics, such as topology changes and the arrival and departure of flows. We introduce *tentative credits* to further improve utilization and better exploit credit paths.

Instead of directly adjusting the pace at which credits are sent (as ExpressPass does), receivers always pace credits at the maximum rate, where regular credits are sent at the target credit rate (§4.3), and the remainder are marked as *tentative credits* to fill the leftover bandwidth. We set a low *tentative credit threshold* in the credit queue (e.g., 2–4 credit packets as in Fig. 7) to accept tentative credits only when the queue is nearly empty, ensuring they do not compete with regular credits in the presence of bottlenecks. Note that this method is similar to Aeolus [21]—which we also utilize (§4.5)—but applied to the credit queues.

For example, in Fig. 8a, two flows (green and orange) are assigned 50% and 33% target credit rates by credit rate control. Clearly, 33% is an undershooting, as the orange flow can actually utilize 50% of the link bandwidth. In this case, regular credits (dark green and dark orange packets) pass through, along with some tentative credits (light orange packets), which successfully compensate for the undershooting. When the green flow completes, as shown in Fig. 8b, additional tentative credits from the orange flow are admitted immediately, achieving full link utilization without waiting for credit rate control to adjust the target rate.

The above examples show how tentative credits complement credit rate control to locally correct suboptimal credit allocation. In Flare, receivers continuously send credits at the maximum pace, preventing credit queues from depleting and ensuring that spare network capacity is effectively utilized.

4.5 Other Optimizations

Starvation prevention. Although probabilistic credit admission (§4.2) provides long-term fairness, short flows can still starve if they traverse long paths. To address this, we design a credit adjustment mechanism that allows flows to “jitter” their *declared* hop count, fooling Eqn. 1 to shift the admission probability. Newly started flows jitter down their hop count to increase the admission probability, while long-running flows jitter up to reduce admission and free up resources for short flows. Note that this adjustment modifies the credit admission probability by embedding the jittered hop count in the packet, without affecting the actual path.

Similar to flow aging [6], we do not assume the flow size is known in advance and instead rely on the amount of received data at the receiver. We define a jitter reference value $Jitter = \frac{data_received}{BDP}$, normalizing received data by the bandwidth-delay product (BDP). Two parameters α and β are set where $Jitter \leq \alpha$ identifies short flows with hop counts decremented by one, and $Jitter \geq \beta$ identifies long flows with hop count incremented by

one. This lightweight mechanism avoids significant instability in probabilistic credit admission. Additionally, a ratio q determines the percentage of packets adjusted.

Sensitivity tests (§6.2) show $\alpha = 4$, $\beta = 16$, and $q = 0.5$ provide the best throughput-latency tradeoff. This setup jitters -1 hop for flows that sent under 4BDP and $+1$ hop for those over 16BDP, only 50% of the time.

Fast start. ExpressPass wastes one RTT of bandwidth after connection setup, delaying short flows and hurting network utilization. To enable fast start, one solution is sending the first BDP of data as “unscheduled” packets without credit scheduling [9, 20, 34]. However, this approach can lead to unpredictable buffering on the data path [21], violating the requirement that the RTT must stay strictly within the time slice duration (§4.1).

To address this, we couple Flare with Aeolus [21]. In Aeolus, first-BDP data packets are sent as “unscheduled”, meaning that they opportunistically utilize spare bandwidth if available, avoiding wasting the first RTT for connection setup. If data queues are already above a small threshold, the packets are proactively dropped. The sender always follows with an ACK request packet, ensuring any lost packets are promptly retransmitted via credits. This design allows Flare to maintain near-zero queue occupancy on the data path while avoiding unnecessary delays during connection startup.

The data-path delay is the sum of the queuing delay (determined by the Aeolus threshold) and the propagation delay, multiplied by the network diameter. Given a target time slice duration (or RTT in Flare), the credit-path delay is RTT minus the data-path delay. The credit queue size can then be derived from the credit-path delay, which is the sum of the credit queuing delay (from the target credit queue size) and propagation delay, multiplied by the network diameter.

5 Implementation

Flare supports both switch- and host-centric RDCNs, where OCSes interconnect switches or hosts. We prototype the more complex switch-centric design on Intel Tofino2 switches and DPDK 23.11 on Mellanox ConnectX-6 NICs. Our implementation includes ~800 lines of code (LoC) for the switch control plane program and scripts, ~900 LoC of P4 code for the switch data plane, and ~2000 LoC of DPDK code.

Flare uses two packet types: DATA and CREDIT. The customized transport header contains these essential fields: `time slice` for time-based routing in Opera (§2.2), `hop count` for probabilistic credit admission (§4.2), `tentative` differentiating tentative and regular credits (§4.4), and `unscheduled` marking unscheduled packets for fast start in Aeolus (§4.5). CREDIT packets are minimum-size Ethernet frames. Flare uses two queues at each link: the credit queue runs at higher priority than the data queue but is rate-limited to $\frac{credit_size}{(credit_size + MTU_size)}$ of bandwidth, which is ~1% of the 40 Gbps bandwidth with jumbo frames in our testbed (§7).

Switches and NICs behave the same to shape credits at *all* links. We implement Flare’s mechanisms (§4) straightforwardly on the DPDK software NICs and highlight challenges with the switch implementation below.

Opera and time-based routing. Though not the main focus, Opera is a prerequisite for Flare. We implement it to the best of our

ability, given its proprietary nature [31]. On our testbed (§7), we emulate the optical fabric using a Tofino2 switch, which relies on its on-chip packet generator to send sync messages to the ToRs at the start of each time slice, updating their *time slice register* with the slice index³.

The Opera routes are precomputed based on the circuit schedule and stored in each source ToR's lookup table to match routes based on the current time slice ts as $\langle ts, src, dst \rangle \rightarrow \langle path \rangle$ entries (Fig. 3b). The action field *path* contains the next-hop egress ports, which are written into the packet using the Strict Source and Record Route (SSRR) option for *source routing* [38]. The source ToR inserts the slice index into the *time slice* field of CREDIT and unscheduled DATA packets, which are then forwarded with source routing. Regular DATA packets copy the *time slice* from their CREDIT packets to maintain path symmetry (§4.1) and are routed based on the copied slice index.

Selective dropping. We realize the two thresholds and queue size for credit queues (Fig. 7) and the Aeolus threshold for data queues using *color-aware dropping*, which supports three dropping thresholds (colors) in addition to full-queue drops. For *credit queues*, we set the queue size to 8 CREDIT packets, a red-color threshold of 12.5% for tentative credits (1 packet), and a yellow-color threshold of 50% (4 packets) for the congestion threshold. We enable dynamic buffering for *data queues* and set the red-color threshold to make the Aeolus threshold 2 MTU-sized packets. DATA packets with unscheduled set and CREDIT packets with tentative set are marked red-colored in the ingress pipeline and dropped after exceeding their thresholds. Next, we detail dropping yellow-colored regular CREDIT packets probabilistically.

Probabilistic admission. With the precomputed Opera paths, we also *precompute* the admission probabilities (Eqn. 1) of every possible hop count (up to 5 hops on our testbed). These probabilities are mapped to a 16-bit range [0,65535], due to the lack of floating-point support on Tofino2, and stored in a table $\langle hop \rangle \rightarrow \langle prob_{16} \rangle$. Each regular CREDIT packet is marked yellow-colored if its table value is smaller than a random 16-bit number generated by the on-chip random number generator. Marked packets are dropped if they exceed the 50% congestion threshold. For example, 1-hop packets map to 65535 and will not be marked yellow-colored thus not dropped until reaching the credit queue size.

6 Simulation

We evaluate Flare using large-scale simulations with production DCN traffic traces. Our analysis includes microbenchmarks to assess Flare's design effectiveness (§6.2), performance comparisons against TDTCP, NDP, ExpressPass, Bolt, and a hop-by-hop baseline (§6.3), and a demonstration of Opera with Flare outperforming Clos-based DCNs (§6.4). We further evaluate implementation feasibility and cross-validate simulation results via testbed experiments in §7.

6.1 Simulation Setup

Network setup. We mirror the original Opera [30] simulation setup on the *htsim* simulator to enable direct comparisons. The simulated Opera topology consists of 108 ToRs, each with 6 uplinks

connecting to 6 108-port OCSeS, and 6 downlinks to hosts, totaling 648 hosts. Each configuration cycle contains 108 time slices. Propagation delay between ToRs is 500 ns (for 100 meters of fiber), while delay within a rack is ignored. Bandwidth at all links is 100 Gbps, with MTU size of 1.5 KB. Consistent with the original Opera evaluation, we also simulate a similar-cost Clos network comprising 126 switches. We use Opera's default single-path multi-hop routing⁴ and run ECMP on Clos.

Time slice duration. We run Opera with two time slice durations: 55 μ s, matching the original Opera evaluation [30], and a shorter 15 μ s, which stresses transport with a stricter queuing delay requirement⁵ (§4.5).

Baseline protocols. We compare Flare against state-of-the-art protocols: **NDP** [20], used by Opera and representative of credit-based transport for non-blocking Clos networks [9, 17, 34]; **ExpressPass** [14], the closest to Flare but without RDCN-specific optimizations; **TDTCP** [13], the latest transport for hybrid electrical-optical RDCNs; **Bolt** [5], the most efficient sub-RTT reactive protocol for traditional DCNs; and **HbH**, a Hop-by-Hop baseline inspired by Shale's traffic control [3], detailed in Appx. C.

We enhance ExpressPass with Aeolus [21] for fast start and integrate TDTCP's reordering detection heuristic [13] into Bolt to prevent spurious retransmissions under topology changes [15]. TDTCP parameters are tuned for Opera (Appx. B). RTO is set to 200 μ s for all protocols except for HbH, which assumes ideal loss recovery with immediate packet retransmission (Appx. C). For Flare, we configure the credit queue size, tentative credit threshold (§4.4), and Aeolus threshold (§4.5) as 60/4/40 and 16/2/8 packets for 55 μ s and 15 μ s time slices, respectively. The credit congestion threshold (§4.2) is set to 50%. We made our best effort to tune the baseline protocols to their best settings based on their original papers, with full details in Appx. §A.

Traffic traces. We run the *web search* [1], *Hadoop* [41], and *RPC* [5] traces widely used in evaluation of modern transport protocols [5, 20, 26, 34]. We scale the traces to reach 30% utilization on the host-to-ToR links (denoted as *30% load*). At 30% load, the network core exceeds 90% utilization. Given the average 3.1-hop path length of Opera's varying expander topology, the loose theoretical load limit would be 32.25% assuming fair bandwidth allocation and uniform traffic. Most experiments use 30% load unless otherwise specified, and we further stress the network to 35% load in some experiments.

6.2 Microbenchmarks

We run microbenchmarks of Flare's key design components using the *web search* trace at 30% load, with a 55 μ s time slice duration, matching the original Opera evaluation [30].

A key contribution of Flare is significantly reducing credit waste by leveraging the dynamic paths of the varying expander. To better explain the results, we quantify credit waste using a metric that measures the extent of waste.

⁴No flow in our traces is large enough to be offloaded to RotorNet's VLB mode (§2.2) without incurring heavy latency degradation. All flows run on the default multi-hop routing, which is Opera's worst-case traffic.

⁵Assuming a 6.2 μ s base RTT (with zero queuing over 5 maximum hops) and a low $\sim 10 \mu$ s total queuing delay, 15 μ s is a tight limit for Opera to still ensure no persistent loss in the reconfigurable network fabric.

³This simple sync scheme achieves $\sim 1.4 \mu$ s accuracy, i.e., latency between switches, sufficient for our 100 μ s time slice duration on the testbed (§7).

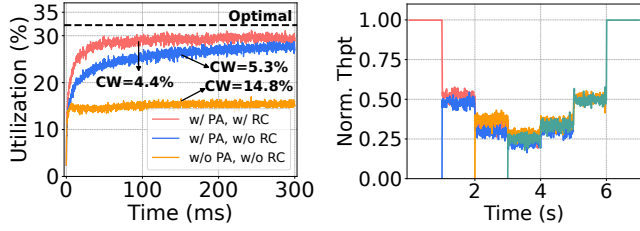


Figure 9: Effect of probabilistic admission (PA) and credit rate control (RC), with credit waste (CW) annotated.

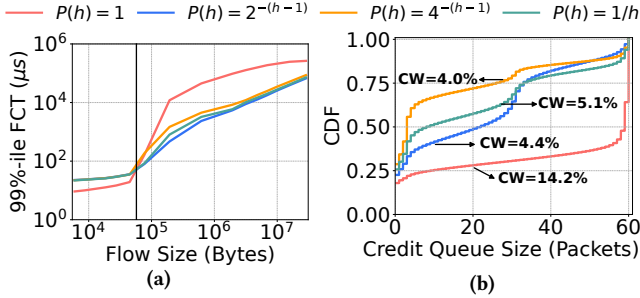


Figure 11: Effect of probability functions on (a) FCTs and (b) credit queue sizes, with credit waste (CW) annotated.

$$\text{credit waste (CW)} = \frac{\sum_{w \in \mathcal{W}} \text{hop}_w}{|\mathcal{C}| \times \text{average_len}} \quad (2)$$

A credit is considered wasted when it traverses one or more links and is dropped before reaching its destination. Intuitively, the greater the distance a credit travels before being dropped, the higher the waste. We denote the set of valid credits sent as $\mathcal{C}$ (i.e., non-tentative as in §4.5), wasted or successfully delivered, and denote wasted credits as $\mathcal{W}$. For each wasted credit $w \in \mathcal{W}$, the wasted bandwidth is characterized by the number of hops it has traversed. Therefore, as shown in Eqn. 2 above, we define credit waste (CW) as the total waste, i.e., the sum of wasted hops, normalized by the product of the total number of credits sent and the average path length in the network.

Near-optimal throughput. Credit-based transport typically offers low latency thanks to minimal buffering [14, 20, 40], but it can suffer from low throughput due to credit waste [14, 34]. Flare addresses this by using probabilistic credit admission (§4.2) and combining credit-rate control tailored for the varying expander (§4.3) with tentative credits (§4.4) to optimize throughput. Fig. 9 shows the throughput (ToR-to-host link utilization) breakdown of these mechanisms relative to the topology’s optimal throughput of 32.25% (§6.1), with credit waste highlighted for each setting.

Without probabilistic admission and its dedicated credit-rate control, Flare behaves similarly to ExpressPass naively applied to the varying expander, achieving only about half of the expected throughput due to an excessive credit waste of 14.8%. Probabilistic admission alone closes the gap towards the optimal throughput, but a rate control system unaware of the varying expander’s dynamic nature still leads to 5.3% waste. Combining probabilistic admission with dedicated credit-rate control and tentative credits, Flare achieves near-optimal throughput with only 4.4% waste.

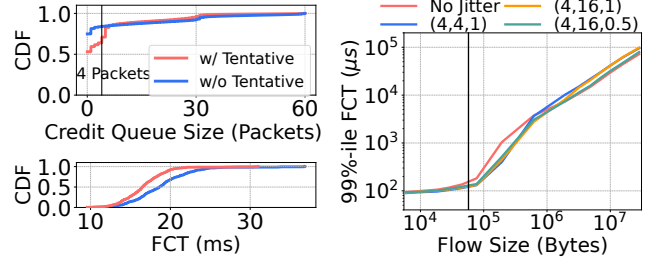


Figure 12: Effect of tentative credits on credit queue sizes (top) and FCT (bottom).

Long-term fairness. Flare prioritizes higher throughput over immediate fairness by using probabilistic admission (§4.2). This approach leverages the long-term fairness of varying expanders, where flows uniformly access short paths across reconfiguration cycles. To validate this, we simulate four 4-second inter-rack incast flows, starting 1 second apart, and measure their throughput (normalized to the maximum 95 Gbps) at cycle-scale (5.9 ms with 55 μ s time slices). As shown in Fig. 10, flows achieve good fairness over time. Yet, slight imbalances occur when certain node pairs experience more favorable paths, causing some flows to receive more bandwidth. This phenomenon is further demonstrated in §7.

Credit admission function. We evaluate different probabilistic admission functions to assess their impact on performance (§4.2). We compare our default function, $P(h) = (\frac{1}{2})^{h-1}$ in Eqn. 1, with: (1) a steeper exponentially decreasing function, $P(h) = (\frac{1}{4})^{h-1}$, to reduce waste by favoring shorter paths, (2) a fairer inversely proportional function, $P(h) = \frac{1}{h}$, and (3) a base case of fair admission, $P(h) = 1$, where all credits are admitted regardless of hop count.

As shown in Fig. 11, our default function achieves a low 99th percentile tail FCT while keeping credit queues near the 30-packet shaping threshold. $P(h) = (\frac{1}{4})^{h-1}$ harms FCT due to reduced queue utilization from overly selective admission. $P(h) = \frac{1}{h}$ lowers FCT but remains higher than the default due to greater credit waste. As expected, fair credit admission, $P(h) = 1$, results in near-full credit queue utilization but causes high credit waste of 14.2% compared to the default’s 4.4%, leading to suboptimal performance.

Tentative credits. Tentative credits are used to prevent under-utilization of credit queues, especially in the presence of sudden path changes or credit rate undershooting (§4.4). To evaluate their impact, we generate sparse traffic with 400 50 MB random inter-rack flows. As shown in Fig. 12 top, tentative credits allow credit queues to contain at least 1-4 credits (below the tentative credit threshold), leading to fuller credit queues and higher utilization. This improves throughput for long flows, reducing tail latency (Fig. 12 bottom).

Credit hop adjustment. Jittering the *declared* hop count helps prevent short-flow starvation and reduces tail latency (§4.5), but excessive deviation from the actual hop count can harm throughput.

We evaluate this in a starvation-prone scenario by halving the initial credit rate for short flows (§4.3) and extending the time slice to 75 μ s, increasing susceptibility to unfavorable configurations.

Fig. 13 shows tail FCTs under different of (α, β, q) settings. We first fix $q=1$ and vary α and β to jitter different portions of traffic. The (4,4,1) setting jitters all flows in their first 4BDP down by one

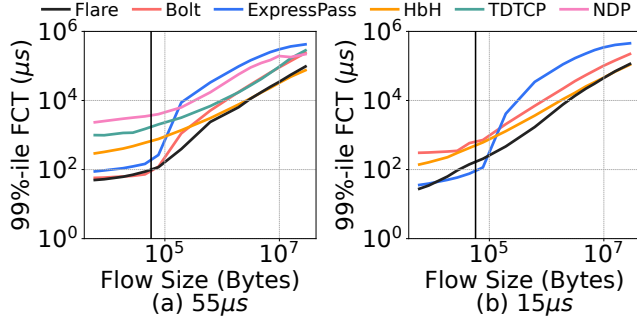


Figure 14: Web search FCTs under different slice durations.

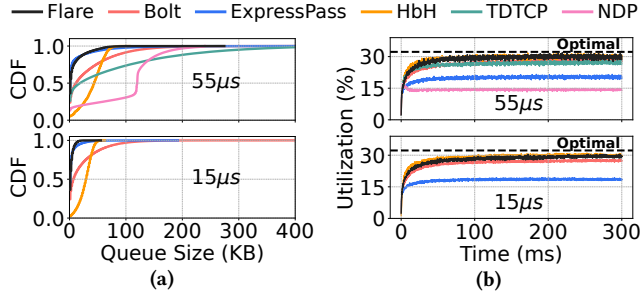


Figure 15: Web search (a) network core data queue sizes and (b) host uplink utilization under different slice durations.

hop, and all flows after 4BDP up by one hop. While reducing FCTs for short flows, this setting also decreases average throughput, penalizing long flow FCTs. The (4,16,1) setting, where the middle portion of traffic between 4 and 16BDP does not jitter, achieves a smoother curve but similar throughput. The best balance is achieved with $q=0.5$, which limits jittering to 50% of the time. It reduces deviation from the credit admission function, particularly for the large portion of traffic from longer flows above the β -threshold. This (4,16,0.5) setting, which we use for our evaluation, reduces tail FCTs by more than 2 $\times$ with negligible impact on throughput (<1% reduction).

6.3 Performance Comparison

We conduct large-scale simulations of various protocols across different traffic traces and time slice durations at 30% traffic load, as detailed in §6.1. Fig. 14-17 present representative results for the 99th percentile tail FCTs, sizes of data queues in the network core, and host-to-ToR link utilizations. The full results, including median FCTs, are in Appx. §D.

Flare. Across Fig.14-17, Flare consistently outperforms all baselines, achieving significantly lower FCTs and higher throughput. It maintains near-optimal throughput absorbing nearly all the loaded traffic, as in Fig. 9, with up to 2 $\times$ and 1.5 $\times$ higher throughput than NDP and ExpressPass, and up to 10 $\times$, 15 $\times$, and 3.5 $\times$ shorter FCTs than ExpressPass, TDTCP, and Bolt.

Low latency, as shown in the FCT figures, results from precise credit-based data scheduling via credit-data path symmetry (§4.1) and starvation prevention for short flows (§4.5), in combination with low buffering (Fig. 15a) and fast start (§4.5). High throughput is primarily driven by probabilistic credit admission (§4.2) and further enhanced by credit rate control (§4.3) and tentative credits

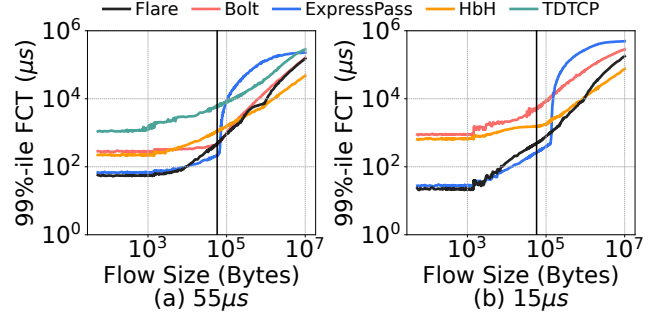


Figure 16: Hadoop FCTs under different slice durations.

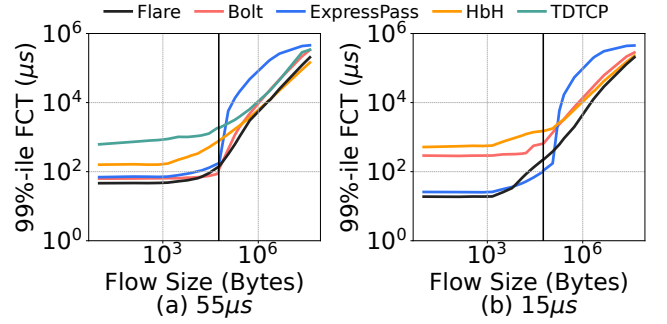


Figure 17: RPC FCTs under different slice durations.

(§4.4). Even with ultra-short 15 μ s time slices, Flare maintains near-full throughput (Fig.15b bottom) and minimal data-path buffering (Fig.15a bottom) without packet losses that plague TDTCP, NDP and Bolt (indicated from severe RTOs in the FCT figures).

Flare vs. NDP. As NDP struggles with multi-bottlenecks in the varying expander (§3.1), it performs poorly in both latency and throughput due to the network collapsing under congestion. As shown in Fig.15b top, its throughput saturates below 15%—half of the input traffic. Meanwhile, Fig.15a top shows most core queues exceeding 100 KB, each adding $\sim 8 \mu$ s queuing delay. In contrast, Flare achieves 2 $\times$ the throughput while keeping 90% of queues below 30 KB. NDP's excessive trimming and poor throughput result in over an order of magnitude higher FCTs than Flare and Bolt. Due to the high resource usage of simulating a collapsing network, we run NDP only for web search traffic (with the least bursty traffic) under 55 μ s time slices, the most forgiving setup.

Flare vs. ExpressPass. ExpressPass effectively controls core congestion and prevents network collapse, as reflected in the low FCTs for short flows. Yet, as discussed in Fig. 9 (§6.2), ExpressPass suffers from high credit waste (14.2% vs. 4.4% in Flare), resulting in up to 10 $\times$ higher FCTs for medium to large flows. Unlike Flare with probabilistic credit admission (§4.2), ExpressPass employs fair tail-drop credit queues that accept credits indiscriminately, even when they are more likely to generate waste. As discussed in §3.2, this method does not perform properly under the rapidly changing network, where relying purely on controlling the credit rate via feedback is mostly ineffective.

Although both ExpressPass and Flare maintain near-zero queuing for over 90% of the samples, ExpressPass exhibits a longer tail in queue sizes (Fig. 15a). For instance, with 15 μ s time slices (Fig. 15a bottom), ExpressPass's tail exceeds 150 KB, despite the 8-packet

(12 KB) Aeolus threshold. This suggests that data packets are not being sent out according to the credit allocation, due to the absence of the credit-data path symmetry explicitly enforced in Flare (§4.1). Note that symmetric routing within a single time slice is enforced in both Flare and ExpressPass, but the latter lacks the mechanism to enforce path symmetry across different time slices.

Flare vs. TDTCP. TDTCP achieves higher throughput than NDP and ExpressPass (Fig. 15b), but still falls short of Flare and Bolt. While tracking congestion states per time slice can improve throughput for long flows, TDTCP’s basic congestion control causes high queueing, as seen in the >200 KB 90th percentile queue size (Fig. 15a). As a result, TDTCP’s tail latency is more than 15× higher than Flare and Bolt due to severe losses. Given that TDTCP already struggles at 55 μ s time slices, we omit results for 15 μ s.

Flare vs. Bolt. Bolt, being the most modern and tailored for highly dynamic traffic, performs well when the time slice duration is relatively high. With the 55 μ s time slice duration, Bolt achieves competitive tail FCTs compared to Flare across all traces (Fig. 14a, 16a, 17a), especially for short flows. However, a noticeable throughput gap remains (Fig. 15a), causing Flare to reduce Bolt’s long flow FCTs by up to 2.47×, 1.91×, and 1.78× in the web search, Hadoop, and RPC traces, respectively. While Bolt’s fast sub-RTT control can handle longer time slice durations, its performance degrades significantly at 15 μ s. Tail FCT rises above the 200 μ s RTO value even for flows under BDP due to high packet loss in the fabric (Fig. 14b, 16b, 17b). This is because Bolt struggles to keep data-path queues low enough to limit end-to-end delays in the 15 μ s slice. Though targeting at near-zero standing queues, its 90th percentile queue size reaches 60.4 KB (~4.8 μ s queueing delay), compared to just 9.0 KB in Flare (Fig. 15a). In contrast, Flare shows no such significant degradation under stricter time slice durations, leading to much lower tail FCTs for short flows compared to Bolt at 15 μ s.

Flare vs. HbH. HbH achieves throughput comparable to Flare (Fig. 15b), reflected in its competitive long-flow FCTs (Fig. 14, 16, 17). However, as discussed in §3.1, the hop-by-hop approach frequently violates the OWD requirement, causing high packet loss—on average 4× higher than the already very lossy TDTCP (Appx. C.3). This stems from HbH’s strictly local admission decisions, which accept packets into queues without accounting for potential congestion on the full end-to-end path, leading to excessive in-flight data. As a result, HbH introduces greater end-to-end delay, driven by both its average queue size being 5× larger than Flare’s (Fig. 15a), even with a strict queue admission policy (§C.2), and by in-flight packets being paused in the middle of the path at bottleneck nodes. The additional delay and packet loss make HbH tail latency up to 7.5× higher for short flows (Fig. 14, 16, 17).

6.4 Performance Gains Over Clos

Opera was originally evaluated against a similar-cost 3:1 oversubscribed Clos network⁶, using NDP on both. The results showed that Opera’s default multi-hop routing mode performed significantly worse than Clos, achieving only 10% maximum traffic load compared to 25% in Clos for the web search workload [30].

⁶Opera has 108 switches and 6 OCSes, Clos has 126 switches. To accommodate the same number of hosts, Clos has to oversubscribe its ToRs.

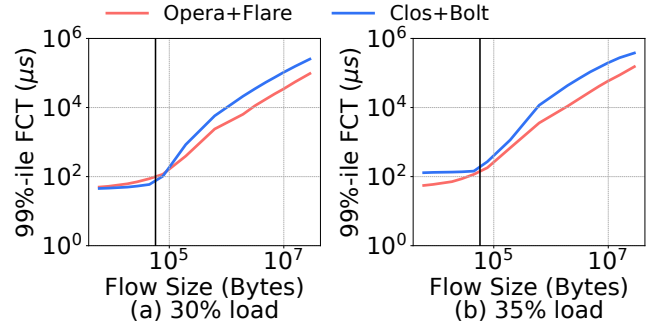


Figure 18: FCTs in Clos vs. Opera at (a) 30%, (b) 35% loads.

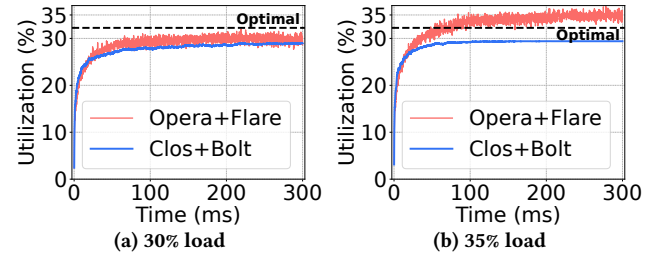


Figure 19: Util in Clos vs. Opera at (a) 30%, (b) 35% loads.

We now revisit this comparison with Flare at our disposal. We pair Opera with Flare, and Clos with Bolt, the most suitable transport for oversubscribed Clos. In this setup, Opera employs single-shortest path routing with 55 μ s time slice duration (same as in [30]), while Clos uses ECMP.

On the web search workload, we run traffic at 30% load (within theoretical capacity of both networks) and 35% load (exceeding capacity). Fig. 18 shows that while both setups have comparable short-flow latency, Flare achieves better long-flow FCT, due to its higher average throughput as shown in Fig. 19a and Fig. 19b. The optimal throughput in Fig. 19 corresponds to the theoretical capacity of Opera’s varying expander of 32.25%, assuming fair bandwidth share among all paths which renders an average path length of 3.1. Flare exploits the reconfigurability of the network by steering bandwidth allocation in favor of shorter paths under heavy load thanks to probabilistic admission (§4.2), thereby surpassing the theoretical capacity. At 35% load, Flare sustains an average throughput higher than 34%, leading to up to a 2.48× reduction in long-flow FCT. Note that the theoretical throughput of 3:1 oversubscribed Clos is 33.3%, which could be achieved with very uniform traffic and packet spraying.

7 Testbed Experiments

We validate Flare’s implementation on our testbed (§5) through targeted experiments. Our testbed consists of three Intel Tofino2 switches and four servers, each equipped with a Mellanox ConnectX-6 dual-port NIC. Two switches emulate four ToRs each, and each server NIC emulates one host per port, totaling eight ToRs. The third switch emulates an OCS interconnecting the ToRs into an 8-node Opera varying expander. We enable 9 KB jumbo frames, with all links at 40 Gbps, and set the credit queue size to 8 packets

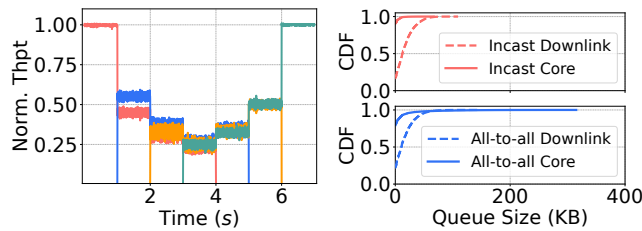


Figure 20: Cycle-level fair share of a 7-to-1 inter-rack incast (top) and all-to-all (bottom) traffic on testbed.

(§5). As the base RTT⁷ ranges from 20 to 60 μ s depending on the path length, we set the time slice duration to a tight 100 μ s.

Long-term fairness. In Fig. 20, we replicate the simulation in Fig. 10 with four 4:1 inter-rack incast flows, normalizing throughput to the maximum 39.8 Gbps. The observed fairness aligns with the simulation results, validating correctness. The first two flows exhibit higher disparity due to the small 8-node testbed, where uniform circuit schedules are harder to maintain, resulting in skewed path lengths—2.25 and 1.625 hops—compared to 2.0 and 1.875 for the last two. At larger scales, path lengths become more uniform, as seen in the 108-ToR simulation, where average path length variance is only 14.6%, compared to 35.2% on the testbed, supporting Flare’s long-term fairness in large-scale RDCNs.

Network queueing. Our switches use in-network telemetry to measure per-hop queueing. Fig. 21 validates credit-scheduling and examines queue behavior under stress using 7-to-1 incast and all-to-all traffic patterns. In the **7-to-1 incast** (seven nodes sending long-running flows to one node for 10 seconds) queues remain controlled, with the 95th downlink queue size at 48 KB. Non-zero queues stem from imperfect pacing in our software host and fluctuating end-to-end delays. Prior work [14] shows RTT variation—driven by asymmetric 1-5 hop paths—increases transient queueing. However, *asymmetry diminishes with scale*, as discussed above, easing deployment concerns. In the **all-to-all** pattern, core queues grow larger than downlink queues under incast, but 95% remain below 20 KB. For context, DCTCP’s recommended K-value for a 40 Gbps network is ~ 360 KB.

8 Discussion

Extended network capacity. We showed Flare can sustain nearly 35% traffic load, even though the Opera network should yield a theoretical maximum capacity of 32.25% (§6.4). To our knowledge, Flare on Opera is the first instance of a transport scheme that enables the network to dynamically inflate its capacity as it saturates. This is possible thanks to its novel probabilistic credit admission system (§4.2), which opportunistically favors shorter paths under bandwidth contention—reducing bandwidth cost per packet. While prior proposals [25] exploit path diversity in RDCNs through static routing to similarly boost capacity, Flare does so dynamically as congestion increases. We believe this opens the door for further exploration of dynamic policies in future transport designs.

Multi-path routing. We evaluate Flare using single-shortest path routing on Opera’s varying expander, following the original

work [30], but this is not a limitation. Flare can support multi-path routing or packet spraying for credits, as long as data packets continue following the reverse credit path. Future work could extend the credit rate control (§4.3) to track multiple paths and dynamically adjust the credit distribution.

Priority-based scheduling. Unlike some other proactive transport [2, 6, 17, 34], Flare does not enforce any flow scheduling policy, e.g., Shortest Remaining Processing Time [34], especially because flow size is assumed to be unknown a priori. Nevertheless, scheduling policies could still be implemented at both the receiver and switches by prioritizing credit packets of higher priority flows.

Intra-rack traffic fairness. Flare bases its long-term fairness on the assumption that all flows uniformly access short paths within a cycle. However, when OCSes connect ToRs, intra-rack flows gain higher admission probability, affecting fairness. To mitigate this, we can generate fictitious path sets per time slice based on the varying expander distribution, embedding dynamic hop counts for intra-rack credits instead of always using the minimum.

Coexistence with background traffic As in the original Opera with NDP, Flare can run alongside background traffic offloaded to RotorLB. In Opera, background traffic runs at a lower priority than multi-hop traffic and is therefore susceptible to starvation under persistent load. A mitigation strategy could be applying weighted priority queueing on the data path, but this risks introducing critical delays for multi-hop packets and increasing fabric packet loss.

Sharing the same credit rate limiting system as ExpressPass, Flare can adopt the FlexPass [28] strategy to more effectively coexist with background traffic. It can further limit its credit rate at links, effectively guaranteeing a portion of the bandwidth for background traffic without impacting latency on the data path. Unused reserved bandwidth can then still be reclaimed opportunistically [28].

9 Conclusion

Opera paved the way to full end-to-end implementations of traffic-oblivious RDCNs [31]. In this paper, we introduce Flare, a credit-based transport that elevates Opera’s varying expander to a high-performance network across all workloads, rivaling and even surpassing the throughput of similar-cost Clos networks. Flare’s innovating credit-based scheduling introduces a unique approach, where data packets are indirectly steered to low-cost paths by shaping credit packets. Credits act as a disposable resource, steering traffic according to some policy—shorter paths in our case—without disrupting in-flight data packets. While we demonstrate a simple stateless shaping scheme at our links, we believe that more complex shaping policies could result in significantly greater performance. We also show how to further leverage path diversity in varying expanders: capacity is not solely determined by the average hop count of a routing scheme, but can be dynamically optimized through transport. We believe this insight could open up new directions of research on unlocking even further performance gains in RDCNs.

Acknowledgments

We thank our shepherd, Keon Jang, and the anonymous reviewers for their insightful comments. We also extend our gratitude to the authors of the Opera paper [30] for sharing their simulator implementation.

⁷Delay measured on an empty network at the receiver from sending a credit to receiving the corresponding data.

References

- [1] Mohammad Alizadeh, Albert Greenberg, David A Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. 2010. Data center tcp (dctcp). In *Proceedings of the ACM SIGCOMM 2010 Conference*. 63–74.
- [2] Mohammad Alizadeh, Shuang Yang, Milad Sharif, Sachin Katti, Nick McKeown, Balaji Prabhakar, and Scott Shenker. 2013. pFabric: minimal near-optimal datacenter transport. In *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM (SIGCOMM '13)*. Association for Computing Machinery, New York, NY, USA, 435–446. <https://doi.org/10.1145/2486001.2486031>
- [3] Daniel Amir, Nitika Saran, Tegan Wilson, Robert Kleinberg, Vishal Shrivastav, and Hakim Weatherspoon. 2024. Shale: A Practical, Scalable Oblivious Reconfigurable Network. In *Proceedings of the ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 449–464. <https://doi.org/10.1145/3651890.3672248>
- [4] Daniel Amir, Tegan Wilson, Vishal Shrivastav, Hakim Weatherspoon, Robert Kleinberg, and Rachit Agarwal. 2022. Optimal oblivious reconfigurable networks. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*. 1339–1352.
- [5] Serhat Arslan, Yulian Li, Gautam Kumar, and Nandita Dukkkipati. 2023. Bolt: Sub-RTT Congestion Control for Ultra-Low Latency. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 219–236.
- [6] Wei Bai, Li Chen, Kai Chen, Dongsu Han, Chen Tian, and Hao Wang. 2015. Information-Agnostic Flow Scheduling for Commodity Data Centers. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*. USENIX Association, Oakland, CA, 455–468. <https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/bai>
- [7] Hitesh Ballani, Paolo Costa, Raphael Behrendt, Daniel Cletheroe, Istvan Haller, Krzysztof Jozwik, Fotini Karinou, Sophie Lange, Kai Shi, Benn Thomsen, et al. 2020. Sirius: A flat datacenter network with nanosecond optical switching. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*. 782–797.
- [8] Joshua L Benjamin, Thomas Gerard, Domanić Lavery, Polina Bayvel, and Georgios Vervas. 2020. PULSE: optical circuit switched data center architecture operating at nanosecond timescales. *Journal of Lightwave Technology* 38, 18 (2020), 4906–4921.
- [9] Qizhe Cai, Mina Tahmasbi Arashloo, and Rachit Agarwal. 2022. dcPIM: Near-optimal proactive datacenter transport. In *Proceedings of the ACM SIGCOMM 2022 Conference*. 53–65.
- [10] Kai Chen, Ankit Singla, Atul Singh, Kishore Ramachandran, Lei Xu, Yueping Zhang, Xitao Wen, and Yan Chen. 2013. OSA: An optical switching architecture for data center networks with unprecedented flexibility. *IEEE/ACM Transactions on Networking* 22, 2 (2013), 498–511.
- [11] Kai Chen, Xitao Wen, Xingyu Ma, Yan Chen, Yong Xia, Chengchen Hu, and Qunfeng Dong. 2015. WaveCube: A scalable, fault-tolerant, high-performance optical data center architecture. In *2015 IEEE Conference on Computer Communications (INFOCOM)*. IEEE, 1903–1911.
- [12] Li Chen, Kai Chen, Zhonghua Zhu, Minlan Yu, George Porter, Chunming Qiao, and Shan Zhong. 2017. Enabling {Wide-Spread} Communications on Optical Fabric with {MegaSwitch}. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. 577–593.
- [13] Shawn Shuoshuo Chen, Weiyang Wang, Christopher Canel, Srinivasan Seshan, Alex C Snoeren, and Peter Steenkiste. 2022. Time-division TCP for reconfigurable data center networks. In *Proceedings of the ACM SIGCOMM 2022 Conference*. 19–35.
- [14] Inho Cho, Keon Jang, and Dongsu Han. 2017. Credit-scheduled delay-bounded congestion control for datacenters. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. 239–252.
- [15] Federico De Marchi, Wei Bai, Jialong Li, and Yiting Xia. 2024. Rethinking Transport Protocols for Reconfigurable Data Centers: An Empirical Study. In *Proceedings of the 1st SIGCOMM Workshop on Hot Topics in Optical Technologies and Applications in Networking*. 7–13.
- [16] Nathan Farrington, George Porter, Sivasankar Radhakrishnan, Hamid Hajabdolali Bazzaz, Vikram Subramanya, Yeshaiah Fainman, George Papan, and Amin Vahdat. 2010. Helios: a hybrid electrical/optical switch architecture for modular data centers. In *Proceedings of the ACM SIGCOMM 2010 Conference*. 339–350.
- [17] Peter X. Gao, Akshay Narayan, Gautam Kumar, Rachit Agarwal, Sylvia Ratnasamy, and Scott Shenker. 2015. pHost: distributed near-optimal datacenter transport over commodity network fabric. In *Proceedings of the 11th ACM Conference on Emerging Networking Experiments and Technologies (CoNEXT '15)*. Association for Computing Machinery, New York, NY, USA, Article 1, 12 pages. <https://doi.org/10.1145/2716281.2836086>
- [18] Monia Ghobadi, Ratul Mahajan, Amar Phanishayee, Nikhil Devanur, Janardhan Kulkarni, Gireja Ranade, Pierre-Alexandre Blanche, Houman Rastegarfar, Madeleine Glick, and Daniel Kilper. 2016. Projector: Agile reconfigurable data center interconnect. In *Proceedings of the 2016 ACM SIGCOMM Conference*. 216–229.
- [19] Prateesh Goyal, Preey Shah, Kevin Zhao, Georgios Nikolaidis, Mohammad Alizadeh, and Thomas E. Anderson. 2022. Backpressure Flow Control. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, Renton, WA, 779–805. <https://www.usenix.org/conference/nsdi22/presentation/goyal>
- [20] Mark Handley, Costin Raiciu, Alexandru Agache, Andrei Voinescu, Andrew W Moore, Gianni Antichi, and Marcin Wójcik. 2017. Re-architecting datacenter networks and stacks for low latency and high performance. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. 29–42.
- [21] Shuihai Hu, Wei Bai, Gaoxiong Zeng, Zilong Wang, Baochen Qiao, Kai Chen, Kun Tan, and Yi Wang. 2020. Aeolus: A Building Block for Proactive Transport in Datacenters. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication (SIGCOMM '20)*. Association for Computing Machinery, New York, NY, USA, 422–434. <https://doi.org/10.1145/3387514.3405878>
- [22] IEEE Standards Association. 2011. *IEEE Standard for Local and metropolitan area networks—Media Access Control (MAC) Bridges and Virtual Bridged Local Area Networks—Amendment 17: Priority-based Flow Control*. Technical Report 802.1Qbb-2011. Piscataway, NJ. <https://doi.org/10.1109/IEEESTD.2011.6032693> Amendment to IEEE Std 802.1Q-2011 as amended by IEEE Std 802.1Qbe-2011 and IEEE Std 802.1Qbc-2011.
- [23] Simon Kassing, Asaf Valadarsky, Gal Shahaf, Michael Schapira, and Ankit Singla. 2017. Beyond fat-trees without antennae, mirrors, and disco-balls. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. 281–294.
- [24] Gautam Kumar, Nandita Dukkkipati, Keon Jang, Hassan M. G. Wassel, Xian Wu, Behnam Montazeri, Yaogong Wang, Kevin Springborn, Christopher Alfeld, Michael Ryan, David Wetherall, and Amin Vahdat. 2020. Swift: Delay is Simple and Effective for Congestion Control in the Datacenter. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication (SIGCOMM '20)*. Association for Computing Machinery, New York, NY, USA, 514–528. <https://doi.org/10.1145/3387514.3406591>
- [25] Jialong Li, Haotian Gong, Federico De Marchi, Aoyu Gong, Yiming Lei, Wei Bai, and Yiting Xia. 2024. Uniform-Cost Multi-Path Routing for Reconfigurable Data Center Networks. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 433–448.
- [26] Yulian Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, Mohammad Alizadeh, et al. 2019. HPCC: High precision congestion control. In *Proceedings of the ACM Special Interest Group on Data Communication*. 44–58.
- [27] Cong Liang, Xiangli Song, Jing Cheng, Mowei Wang, Yashe Liu, Zhenhua Liu, Shizhen Zhao, and Yong Cui. 2024. NegotiaToR: Towards A Simple Yet Effective On-demand Reconfigurable Datacenter Network. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 415–432.
- [28] Hwijoon Lim, Jaehong Kim, Inho Cho, Keon Jang, Wei Bai, and Dongsu Han. 2023. FlexPass: A Case for Flexible Credit-based Transport for Datacenter Networks. In *Proceedings of the Eighteenth European Conference on Computer Systems (EuroSys '23)*. Association for Computing Machinery, New York, NY, USA, 606–622. <https://doi.org/10.1145/3552326.3587453>
- [29] Hong Liu, Ryohei Urata, Kevin Yasumura, Xiang Zhou, Roy Bannon, Jill Berger, Pedram Dashti, Norm Jouppi, Cedric Lam, Sheng Li, et al. 2023. Lightwave fabrics: at-scale optical circuit switching for datacenter and machine learning systems. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 499–515.
- [30] William M Mellette, Rajdeep Das, Yibo Guo, Rob McGuinness, Alex C Snoeren, and George Porter. 2020. Expanding across time to deliver bandwidth efficiency and low latency. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. 1–18.
- [31] William M. Mellette, Alex Forencich, Rukshani Athapathu, Alex C. Snoeren, George Papan, and George Porter. 2024. Realizing RotorNet: Toward Practical Microsecond Scale Optical Networking. In *Proceedings of the ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 392–414. <https://doi.org/10.1145/3651890.3672273>
- [32] William M Mellette, Rob McGuinness, Arjun Roy, Alex Forencich, George Papan, Alex C Snoeren, and George Porter. 2017. Rotornet: A scalable, low-complexity, optical datacenter network. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. 267–280.
- [33] Radhika Mittal, Vinh The Lam, Nandita Dukkkipati, Emily Blem, Hassan Wassel, Monia Ghobadi, Amin Vahdat, Yaogong Wang, David Wetherall, and David Zats. 2015. TIMELY: RTT-based Congestion Control for the Datacenter (SIGCOMM '15). Association for Computing Machinery, New York, NY, USA, 537–550. <https://doi.org/10.1145/2785956.2787510>
- [34] Behnam Montazeri, Yilong Li, Mohammad Alizadeh, and John Ousterhout. 2018. Homa: A receiver-driven low-latency transport protocol using network priorities. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*. 221–235.
- [35] Matthew K. Mukerjee, Christopher Canel, Weiyang Wang, Daehyeok Kim, Srinivasan Seshan, and Alex C. Snoeren. 2020. Adapting TCP for Reconfigurable Datacenter Networks. In *17th USENIX Symposium on Networked Systems Design*

- and Implementation (NSDI 20). USENIX Association, Santa Clara, CA, 651–666. <https://www.usenix.org/conference/nsdi20/presentation/mukerjee>
- [36] Jonathan Perry, Amy Ousterhout, Hari Balakrishnan, Devavrat Shah, and Hans Fugal. 2014. Fastpass: a centralized "zero-queue" datacenter network. In *Proceedings of the 2014 ACM Conference on SIGCOMM (SIGCOMM '14)*. Association for Computing Machinery, New York, NY, USA, 307–318. <https://doi.org/10.1145/2619239.2626309>
 - [37] George Porter, Richard Strong, Nathan Farrington, Alex Forencich, Pang Chen-Sun, Tajana Rosing, Yeshaiah Fainman, George Papen, and Amin Vahdat. 2013. Integrating microsecond circuit switching into the data center. *ACM SIGCOMM Computer Communication Review* 43, 4 (2013), 447–458.
 - [38] Jon Postel. 1981. RFC0791: Internet protocol. (1981).
 - [39] Leon Poutievski, Omid Mashayekhi, Joon Ong, Arjun Singh, Mukarram Tariq, Rui Wang, Jianan Zhang, Virginia Beauregard, Patrick Conner, Steve Gribble, et al. 2022. Jupiter evolving: transforming google's datacenter network via optical circuit switches and software-defined networking. In *Proceedings of the ACM SIGCOMM 2022 Conference*. 66–85.
 - [40] Konstantinos Prasopoulos, Edouard Bugnion, and Marios Kogias. 2023. SIRD: A Sender-Informed, Receiver-Driven Datacenter Transport Protocol. *arXiv preprint arXiv:2312.15403* (2023).
 - [41] Arjun Roy, Hongyi Zeng, Jasmeet Bagga, George Porter, and Alex C. Snoeren. 2015. Inside the Social Network's (Datacenter) Network (SIGCOMM '15). Association for Computing Machinery, New York, NY, USA, 123–137. <https://doi.org/10.1145/2785956.2787472>
 - [42] Asaf Valadarsky, Gal Shahaf, Michael Dinitz, and Michael Schapira. 2016. Xpander: Towards optimal-performance datacenters. In *Proceedings of the 12th International Conference on emerging Networking EXperiments and Technologies*. 205–219.
 - [43] Guohui Wang, David G Andersen, Michael Kaminsky, Konstantina Papagiannaki, TS Eugene Ng, Michael Kozuch, and Michael Ryan. 2010. c-Through: Part-time optics in data centers. In *Proceedings of the ACM SIGCOMM 2010 Conference*. 327–338.
 - [44] Tegan Wilson, Daniel Amir, Nitika Saran, Robert Kleinberg, Vishal Shrivastav, and Hakim Weatherspoon. 2024. Breaking the vlb barrier for oblivious reconfigurable networks. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*. 1865–1876.
 - [45] Yiting Xia, Xiaoye Steven Sun, Simbarashe Dzinamarira, Dingming Wu, Xin Sunny Huang, and TS Eugene Ng. 2017. A tale of two topologies: Exploring convertible data center network architectures with flat-tree. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. 295–308.
 - [46] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. 2015. Congestion Control for Large-Scale RDMA Deployments. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication (SIGCOMM '15)*. Association for Computing Machinery, New York, NY, USA, 523–536. <https://doi.org/10.1145/2785956.2787484>

Appendix

Appendices are supporting material that has not been peer-reviewed.

A Simulation Parameter Settings

Table 1: Full settings for all transport baselines.

Protocol	Parameters	
all	100 Gbps links, 1.5KB MTU 500ns inter-rack delay, 0 intra-rack delay 57440 BDP/initial WND 900 KB max queue size	
/	55 μ s time slice duration	15 μ s time slice duration
Flare	60 pkt credit qsize 30 pkt shaping thresh 40 pkt Aeolus thresh 1.0 <i>winit</i> 0.1 <i>target_loss</i>	16 pkt credit qsize 8 pkt shaping thresh 8 pkt Aeolus thresh 1.0 <i>winit</i> 0.1 <i>target_loss</i>
Bolt	1 MTU <i>CCThresh</i> 1 MTU max <i>sm_token</i>	1 MTU <i>CCThresh</i> 1 MTU max <i>sm_token</i>
ExpressPass	80 pkt credit qsize 40 pkt Aeolus thresh 0.0625 <i>winit</i> 0.1 <i>target_loss</i>	20 pkt credit qsize 8 pkt Aeolus thresh 0.0625 <i>winit</i> 0.1 <i>target_loss</i>
NDP	80 pkt data/credit qsize	/
TDTCP	14360 initial WND 40 ECN	/
HbH	5D 10P	2D 6P

In Table 1, we present the full settings for all transport baselines (§6.1).

B Bolt vs. TDTCP

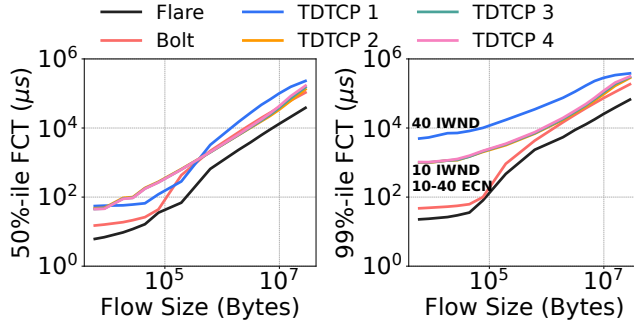


Figure 22: TDTCP on web search FCTs under 55 μ s time slice duration. TDTCP 1 uses 40IWND and 40 packets ECN. TDTCP 2-4 use 40IWND and respectively 40, 20, and 10 ECN.

We implement TDTCP as in the original work [13], but base it on DCTCP [1] instead of TCP Cubic in order to try and keep queue sizes lower and prevent excessive fabric loss in Opera. Fig. 22 shows the results for median and tail FCT. TDTCP performs significantly worse than both Flare and Bolt at the tail, regardless of setting. Differently from Flare and Bolt, it cannot sustain a full BDP (40 packets) of initial window (IWND) without heavy packet loss, which results in up to 5.46 $\times$ higher FCT compared to a more conservative setting of 10 packets. Running the more conservative initial window across ECN settings of 10, 20, and 40, TDTCP still suffers from frequent losses with flows below BDP having 15 $\times$ higher FCT than

Flare and Bolt. TDTCP also has 8% lower throughput than Flare, resulting in 4.24 $\times$ and 2.78 $\times$ higher long flow FCT than Flare and Bolt, respectively.

C Hop-by-Hop Baseline

As argued in §3.1, end-to-end traffic control is the most effective approach to bounding the maximum OWD in Opera’s varying expanders. However, different RDCN designs adopt different transport mechanisms based on their architectural requirements. A prominent example is nanosecond-switched architectures such as Sirius [7] and Shale [3], which employ purely hop-by-hop traffic control tailored to their ultra-short time slices where packets are intermittently transferred hop-by-hop rather than end-to-end.

In order to empirically evaluate the effectiveness of hop-by-hop versus end-to-end control in Opera’s varying expanders, we implement a baseline transport mechanism inspired by Shale, which we refer to as **HbH** (Hop-by-Hop). However, as detailed below, Shale’s original mechanism is designed to operate with nanosecond-scale time slices and thus does not directly translate well to Opera. We adapt this hop-by-hop system to function on Opera and optimize its performance to the best of our capabilities to enable a meaningful comparison.

C.1 Adapting to Opera

Token exchange. In Shale, data packets are sent from one node only when a token is granted from the successive node (i.e., a form of hop-by-hop proactive transport, as in §3.1). Because each time slice is only long enough to allow the transfer of a single packet, token-data exchanges are paced by the passing of time slices, with one exchange per slice. This exchange model, however, does not map well to Opera, where time slices span microseconds. Attempting to pace token exchanges for multi-hop paths at Opera’s time slice granularity would introduce excessive latency, undermining the benefits of low-latency multi-hop routing. To adapt hop-by-hop control to Opera, we implement a proactive variant with *continuous exchanges*, decoupled from the passing of time slices. In this model, a receiving queue continuously pulls packets from sending queues as long as the optical circuit is up and the following *invariants* are satisfied.

Queue invariants. We follow Shale’s approach in limiting the number of packets in a queue intended for the same destination node to an invariant *D*. This approach serves to bound queue occupancy due to incast. However, *D* alone does not provide a sufficiently tight bound to the number of sent-out packets, as evaluated in Appx. C.2. To address this, we introduce an additional invariant *P*, which limits the number of packets in a queue that are bound for the same next-hop egress port. This second invariant helps control packet buildup when multiple flows to different destinations share a queue on the path. We avoid enforcing a maximum queue size invariant, which in the context of expander-based topologies can introduce cyclic dependencies and lead to deadlocks [19].

Queueing model. Because extracting the first packet in a queue may not satisfy the invariant, other packets can be stuck due to head-of-line blocking, reducing link utilization. To mitigate this, we adopt Shale’s *Push-In Extract-Out* (PIEO) queue model. Under this model, the first packet that satisfies the invariant may be extracted

from any position in the sending queue without incurring additional latency.

Ideal loss recovery. As explained in §3.1, HbH, as a purely hop-by-hop approach, cannot reliably constrain the OWD of packets to prevent loss in the fabric. This results from the hop-by-hop approach lacking an end-to-end view of congestion and bottlenecks, which can lead to excess packets being sent out into the network. These excess packets then require extended buffering while waiting to be pulled by the next queue, possibly missing the window of time to traverse their scheduled route. This leads to packets being forwarded to circuits that are either down or misdirected.

We implement HbH with *ideal recovery*. This means that, in the simulator, each lost packet is immediately queued up for retransmission at the source, preventing spurious or delayed retransmissions. This should provide a view of HbH's performance with the recovery overhead excluded. We evaluate loss more in depth in Appx. C.3.

Ideal token exchange. Similarly, we omit actual token exchanges in the simulator. Instead, queues use the global simulator state to maintain the invariants at all times when transmitting packets. This simulates an idealized version of the system where nodes can exchange queue information without any overhead or error.

C.2 Parameter Tuning

In this section, we tune D and P for HbH to optimize its performance for evaluation in §6.3. We use FCT and the amount of packet loss as indicators of performance, and demonstrate the results of the parameter tuning procedure on the web search trace at 30% load with 55 μ s time slice duration, as done in microbenchmarks (§6.2). Other traces yield similar trends.

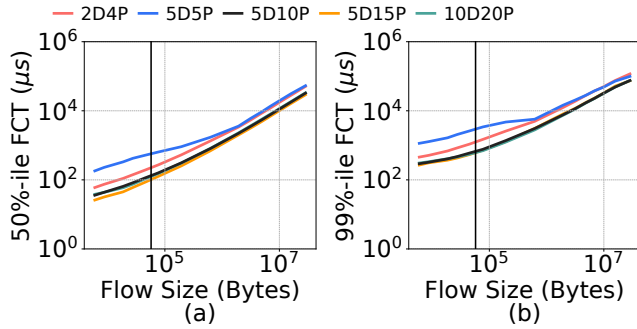


Figure 23: Web search FCTs under 55 μ s time slices for different values of F and P .

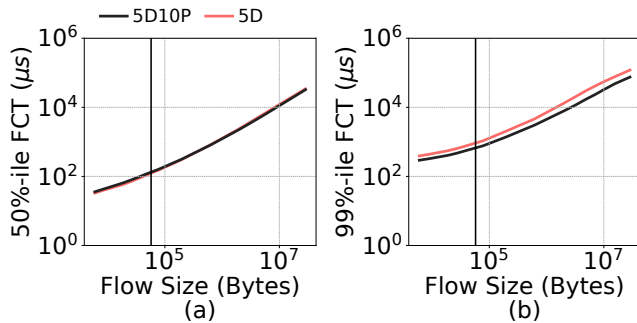


Figure 24: Web search FCTs under 55 μ s time slices with and without the P invariant.

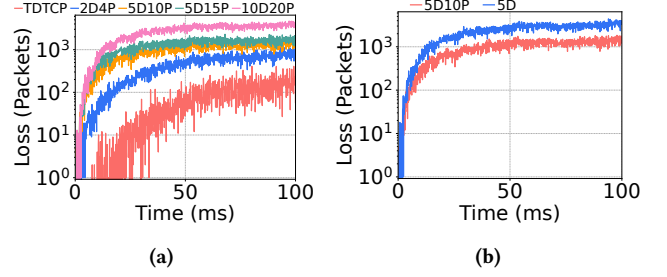


Figure 25: Web search loss under 55 μ s time slices with (a) different values of P and F , and (b) with and without the P invariant.

Tuning D and P . Ideally, D should allow a sufficient number of in-flight packets between nodes to ensure maximum throughput in simple scenarios (e.g., a single flow). In our simulated network with 500 ns propagation and 120 ns transmission delay this corresponds to approximately five MTU-sized packets. Starting from a 5D configuration, we test half and double this value, adjusting P proportionally. Intuitively, tighter values should help bound the OWD, but also lead to a throughput decrease. In the following, we show our results for the 55 μ s time slices, and repeat the process for the 15 μ s time slices. Our final values are 5D10P for 55 μ s slices, and 2D6P for 15 μ s (as in Table 1).

In Fig. 23 we show the median and tail FCT for the different configurations, and in Fig. 25a the corresponding packet loss over time. We find that the 5D10P configuration achieves the best overall latency and the lowest amount of packet loss. While decreasing D and P reduces packet loss, it also inflates overall latency. Conversely, increasing D and P yields minimal improvements in latency while incurring additional packet loss.

Necessity of P . We argue that an additional invariant P , which sets a tighter bound on the number of packets sent into the fabric, is necessary to prevent excessive loss in Opera. We compare our chosen setting (5D10P) against the same setting without the P invariant. We plot the median and tail FCT of the two setups in Fig. 24, and the number of lost packets over time in Fig. 25b. The absence of the P invariant results in inflated FCT for all flows and leads to 2.5 $\times$ more lost packets on average compared to when the P invariant is active.

C.3 A Deeper Dive Into Loss

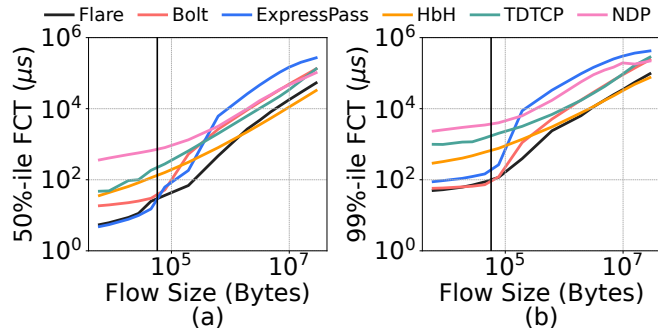
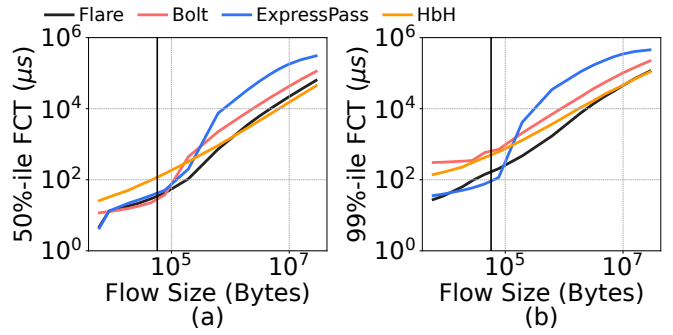
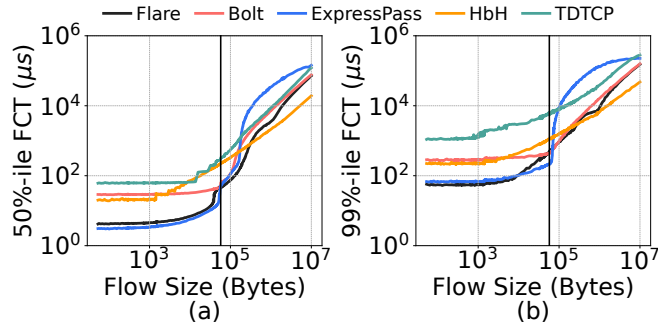
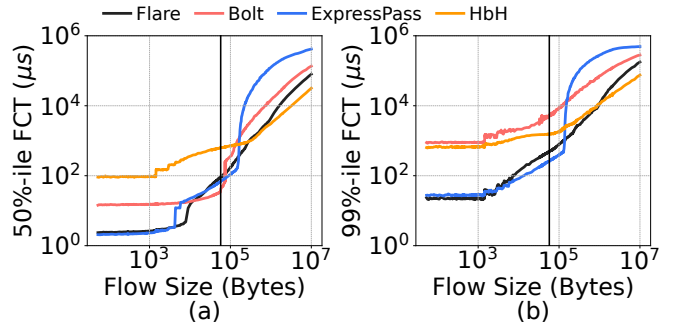
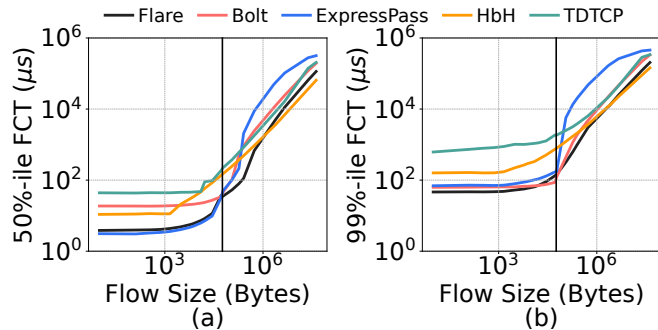
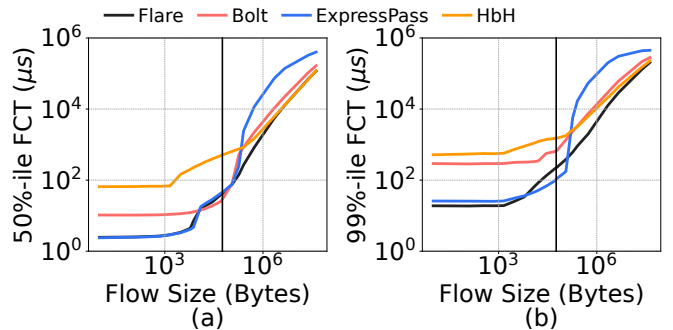
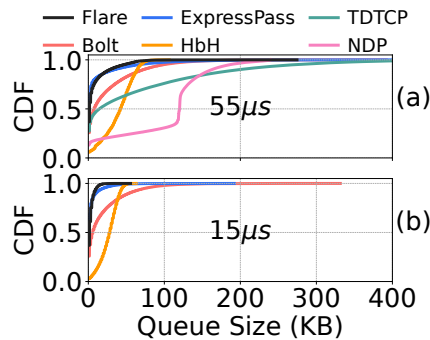
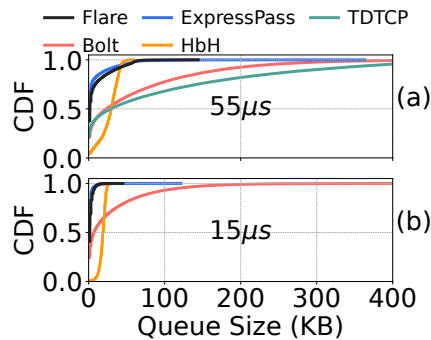
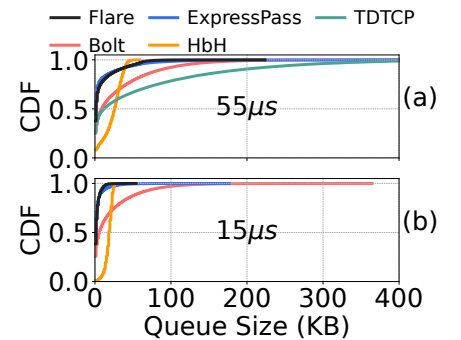
In our performance comparison experiments (§6.3), we have evaluated the performance of HbH using ideal recovery against the other baselines. In this section, we want to note that ideal recovery masks the impact packet loss can have on real, imperfect recovery systems. Even after our parameter tuning, it is hard to bound the OWD using purely hop-by-hop control, which leads to high packet loss in HbH. We put this amount of packet loss into context by comparing it to TDTCP (which we also tune in Appx. B), as we find it the most lossy among the end-to-end baselines (excluding NDP, which uses packet trimming).

In Fig. 25a, we plot the number of lost packets for TDTCP alongside HbH with various D and P settings. Even after tuning, our chosen HbH setting experiences 4 $\times$ higher packet loss than TDTCP,

which is using rudimentary, albeit end-to-end, ECN-based congestion control. This is because HbH regulates traffic on a per-hop basis, rather than bounding the total amount of in-flight traffic. On Opera's varying expander, this results in severe packet loss due to circuit reconfiguration, as hop-by-hop traffic control does not constrain per-packet OWD as effectively as end-to-end approaches.

D Full Simulation Results

In Fig. 26 and Fig. 27, we show web search FCTs under $55\text{ }\mu\text{s}$ and $15\text{ }\mu\text{s}$ time slice durations. The same results for Hadoop and RPC are shown in Fig. 28-Fig. 31. Network core data queue sizes under both time slice durations are presented in Fig. 32 for web search, and in Fig. 33 and Fig. 34 for Hadoop and RPC. Host uplink utilization results are shown in Fig. 35 for web search, and Fig. 36 and Fig. 37 for Hadoop and RPC.

Figure 26: Web search FCTs under 55 μ s time slice duration.Figure 27: Web search FCTs under 15 μ s time slice duration.Figure 28: Hadoop FCTs under 55 μ s time slice duration.Figure 29: Hadoop FCTs under 15 μ s time slice duration.Figure 30: RPC FCTs under 55 μ s time slice duration.Figure 31: RPC FCTs under 15 μ s time slice duration.Figure 32: Web search network core data queue sizes at (a) 55 μ s, (b) 15 μ s time slice duration.Figure 33: Hadoop network core data queue sizes at (a) 55 μ s, (b) 15 μ s time slice duration.Figure 34: RPC network core data queue sizes at (a) 55 μ s, (b) 15 μ s time slice duration.

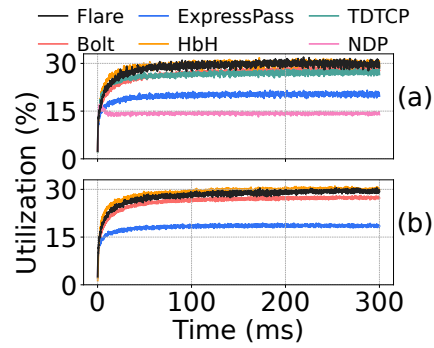


Figure 35: Web search host uplink utilization at (a) $55\ \mu\text{s}$, (b) $15\ \mu\text{s}$ time slice duration.

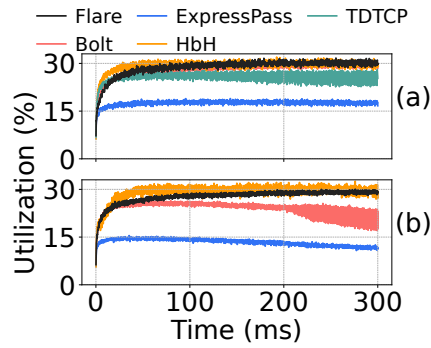


Figure 36: Hadoop host uplink utilization at (a) $55\ \mu\text{s}$, (b) $15\ \mu\text{s}$ time slice duration.

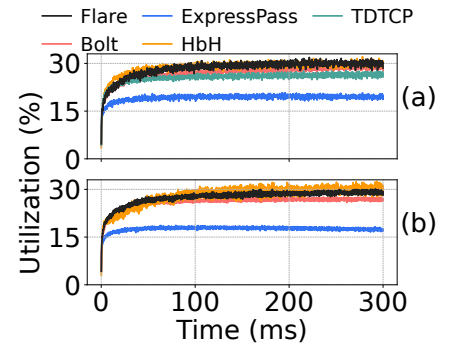


Figure 37: RPC host uplink utilization at (a) $55\ \mu\text{s}$, (b) $15\ \mu\text{s}$ time slice duration.